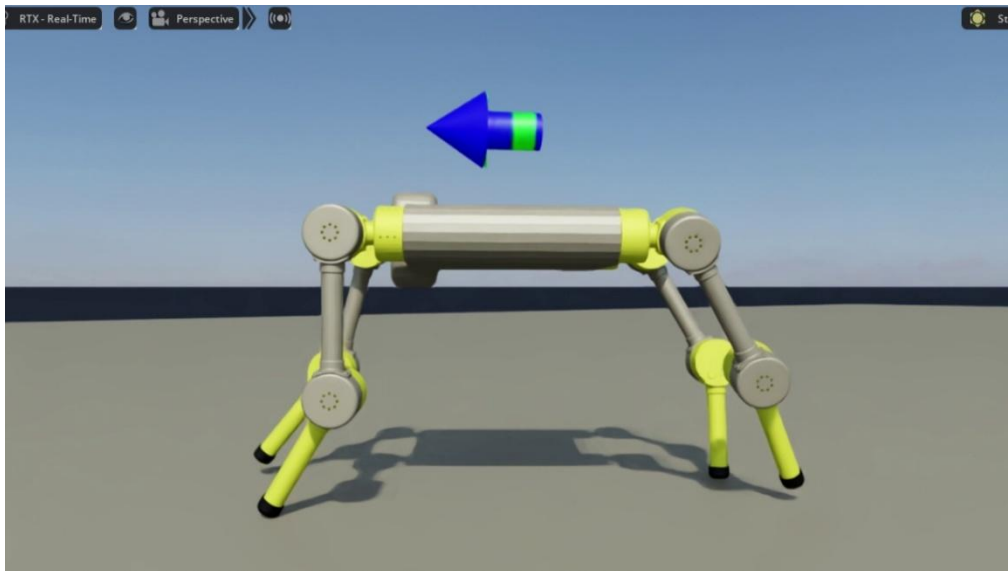


UNIVERSITY OF SALFORD
School of Computing Science and Engineering



University of
Salford
MANCHESTER

Digital Twin Development and Reinforcement
Learning for Robust Quadruped Locomotion in
NVIDIA Isaac Lab



MSc in Robotics and Automation
April 2026

Stephan Bakun

Supervisor: Dr. Saber Mahboubi Heydarabad

University of Salford

School of Computing Science and Engineering

Student's Name: Stephan Bakun

Course Code:

Project Title: Digital Twin Development and Reinforcement Learning for Robust Quadruped Locomotion in NVIDIA Isaac Lab

I certify that this report is my own work. I have properly acknowledged all material that has been used from other sources, references, etc.

Student's Signatures: 

Date: 24.04.2026

Official stamp:

Submission date (to be entered by relevant School Office staff):

ABSTRACT

Deploying autonomous quadrupeds in hazardous environments, such as nuclear decommissioning zones, is hindered by the sim-to-real gap, where simulated locomotion policies often cause catastrophic hardware failure. This dissertation investigates the development of a transfer-ready digital twin to safely engineer a robust locomotion controller for a custom 64.36 kg quadruped. Utilizing NVIDIA Isaac Lab and Proximal Policy Optimization (PPO), the methodology prioritizes strict physical realism by enforcing absolute actuator torque (229.0 Nm) and velocity limits alongside comprehensive domain randomization. The results demonstrate successful policy convergence, yielding a stable 0.6 m/s trot. Faced with extreme kinematic stress, the agent synthesized a highly resilient gait, maintaining continuous dynamic stability and seamless momentum transfer while successfully recovering from 1.5 m/s lateral perturbations without violating mechanical constraints. Ultimately, this digital twin decouples software control from hardware availability, establishing a validated "simulate-then-build" paradigm that ensures physical viability prior to real-world deployment.

ACKNOWLEDGMENTS

I would like to sincerely thank Dr. Heydarabad for introducing me to this fascinating intersection of machine learning and real-world robotics, and for providing me with the opportunity to contribute to such a cutting-edge technological development. His guidance throughout this project has been genuinely invaluable.

I would also like to express my deepest gratitude to my mother. Her emotional and financial support made the completion of this work possible. That she provided this support while living alone in a foreign country, under the shadow of conflict, makes her sacrifice immeasurable. This work is as much hers as it is mine.

TABLE OF CONTENTS

1. Introduction	- 1 -
1.1 Problem	- 1 -
1.2 Aims and Objectives	- 2 -
1.3 Project Plan	- 3 -
1.4 Dissertation Outline	- 5 -
2. Background	- 6 -
2.1 Key Concepts	- 6 -
2.1.1 Reinforcement Learning and PPO	- 6 -
2.1.2 Legged Locomotion and Biomechanics	7
2.1.3 The Sim-to-Real Gap	8
2.2 Literature Review	10
2.2.1 Introduction	- 10 -
2.2.2 Digital Twin Workflow for Locomotion	11
2.2.3 Foundations: Balance, Terrain Interaction, and Robustness	- 10 -
2.2.4 Deep RL for Quadruped Locomotion: Problem Formulation and Control Interfaces	- 10 -
2.2.5 Robustness in Simulation: Domain Shifts, Randomization, and What to Model	- 10 -
2.2.6 Partial Observability, Memory, and Estimation	- 10 -
2.2.7 Robustness Beyond Nominal Walking: Skills, Recovery, and Composition	21
2.2.8 Alternatives and Complements to Vanilla PPO	23

2.2.9	Synthesis and Dissertation Gap	25
3.	Model Architecture.....	- 27 -
2.1	Model Description	- 27 -
3.2	Model Analysis	- 31 -
3.3	Implementation	- 38 -
4.	Experimental Results	- 47 -
4.1	Methodology	- 47 -
4.2	Results	- 49 -
4.3	Analysis.....	62
5.	Conclusions.....	73
5.1	Summary	73
5.2	Discussion	74
5.2.1	Contributions	74
5.2.2	Impact	75
5.2.3	Significance	76
5.3	Future work	76

LIST OF FIGURES

Figure 1: MSc Project Gantt Chart.	3
Figure 2: A Base Link.	28
Figure 3: A Hip Link.	28
Figure 4: A Thigh Link.	28
Figure 5: A Calf Link.	28
Figure 6: A Foot Link.	29
Figure 7: The quadruped rendered as a USD asset within NVIDIA Isaac Lab.	30
Figure 8: The actuator control pipeline.	34
Figure 9: The software architecture of the digital twin.	39
Figure 10: UML class inheritance diagram.	44
Figure 11: The mean episodic reward over the 1500-iteration training cycle.	49
Figure 12: The mean episode length over the 1500-iteration training cycle.	50
Figure 13: The forward velocity reward over the 1500-iteration training cycle.	51
Figure 14: The feet air-time reward over the 1500-iteration training cycle	52
Figure 15: The actuator torque regularization penalty over the 1500-iteration training cycle.	54
Figure 16: The action rate penalty over the 1500-iteration training cycle	54
Figure 17: The undesired contacts penalty over the 1500-iteration training cycle	55
Figure 18: The hip-roll constraint penalty over the 1500-iteration training cycle	56
Figure 19: The chassis orientation penalty over the 1500-iteration training cycle.	57
Figure 20: Time-series push recovery telemetry	59
Figure 21: Visual telemetry of the converged policy	60

Figure 22: Visual telemetry demonstrating successful dynamic recovery and target speed re-establishment 61

Figure 23: Visual telemetry documenting the kinematic edge-cases where a lateral impulse induces a terminal capsizing failure 61

LIST OF TABLES

Table 1: Kinematic and inertial properties of the quadruped robot..... 29

1. INTRODUCTION

1.1 Problem

Human operators are frequently required to perform critical tasks in environments that are inherently hazardous, where exposure poses severe short- and long-term health risks. A primary example is the inspection and decommissioning of nuclear facilities. In these settings, unstructured obstacles, uncertain terrain, and hazardous ambient conditions present significant threats to human safety. Consequently, deploying autonomous robotic agents to conduct these missions is highly desirable to mitigate risk and prevent potential loss of life.

However, deploying robots in such unstructured, cluttered environments presents a profound mobility challenge. Traditional wheeled or tracked vehicles struggle to navigate terrain discontinuities, debris, and stairs. While quadrupedal robots offer the necessary agility to traverse these hazards, developing reliable, robust locomotion controllers for them remains exceptionally difficult.

To overcome this, engineers increasingly rely on physics simulators and Deep Reinforcement Learning (DRL) to synthesize gaits. Yet, a critical "reality gap" exists: locomotion policies that succeed in idealized simulations frequently degrade or fail catastrophically when transferred to physical hardware. This is often due to unmodeled actuator torque limits, inertial kinematics, and mechanical delays.

The core problem this dissertation addresses is the need to construct a functioning, reliable digital twin for a custom quadruped robot intended for inspection as part of nuclear decommissioning missions. In this context, the digital twin serves as more than a virtual model; it is a rigorous physics simulation used to safely train the artificial "brain" (the control policy) that will ultimately dictate how the physical robot balances and walks. Because training Reinforcement Learning (RL) policies directly on physical hardware in these environments is dangerous and impractical, this simulation-first workflow is required. However, to successfully bridge the reality gap and enable real-world deployment, this digital twin must strictly enforce physical limits—such as actuator penalties and mechanical constraints—ensuring that the synthesized control policies are mechanically viable and safe for the physical robot.

1.2 Aims and Objectives

The primary aim of this dissertation is to develop this simulation-based digital twin within NVIDIA Isaac Lab to train and evaluate a reinforcement learning locomotion controller. By systematically implementing a complete RL pipeline—from URDF asset integration to baseline training and strict policy evaluation—the project establishes a rigorous framework for stress-testing the custom robot under realistic physical constraints. Ultimately, this digital twin serves as a diagnostic tool, demonstrating the critical value of simulation-first validation prior to hardware deployment.

1.3 Project Plan

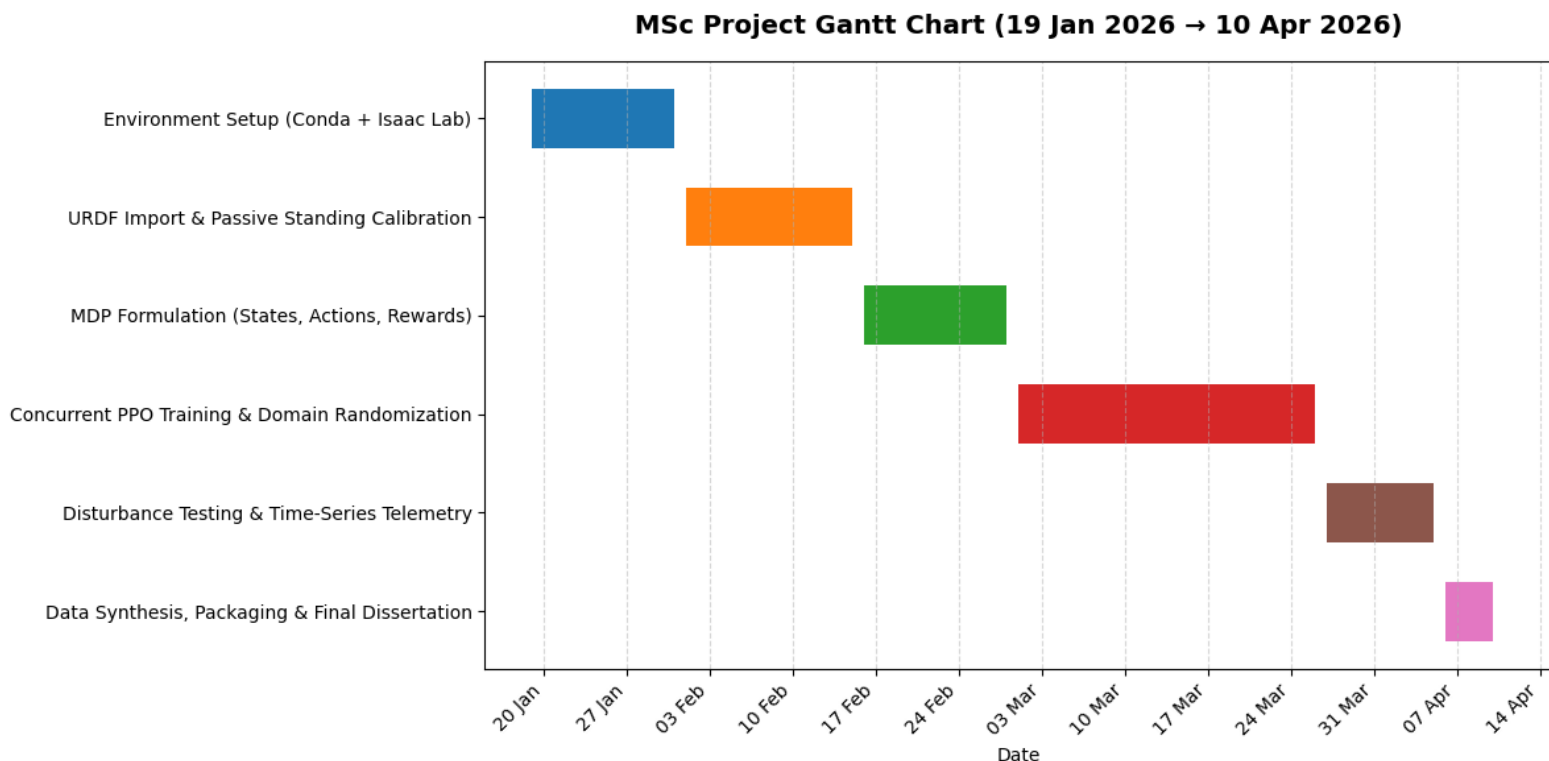


Figure 1: MSc Project Gantt chart detailing the **12-week** development timeline.

Project Plan Discussion

The project was executed over an intensive 12-week schedule, running from mid-January to early April 2026, structured to ensure a systematic progression from foundational software configuration to final policy evaluation. While the initial project proposal scoped for multi-terrain training and ablation studies, the empirical reality of the training process necessitated a more targeted approach. The timeline was strictly managed across the following core phases:

- **Phase 1: Environment & Asset Configuration (Weeks 1-4):** The initial phase was dedicated to establishing the core software stack, including Conda environment setup and

completing NVIDIA Isaac Sim/Lab tutorials. Following this, the physical CAD model was converted and imported via URDF into the Isaac environment, culminating in the successful configuration of a stable, passive standing posture.

- **Phase 2: Task Formulation & Concurrent Training (Weeks 5-10):** The middle phase transitioned into active reinforcement learning. The continuous control environment was explicitly defined (mapping observations, actions, rewards, and termination conditions). Once the Markov Decision Process (MDP) was formalized, the PPO algorithm was deployed. Crucially, rather than training a baseline and randomizing it later, the Domain Randomization pipeline (friction, mass, and actuator variance) was actively injected *alongside* the baseline training. This concurrent approach prevented the agent from overfitting to idealized simulation physics and forced the immediate synthesis of a robust, hardware-ready trotting gait.
- **Phase 3: Robustness Evaluation (Week 11):** With a stable, randomized policy achieved, the focus shifted to empirical testing. The policy was systematically evaluated using an isolated time-series disturbance test to measure dynamic push recovery under peak physical loads.
- **Phase 4: Documentation (Week 12):** The final week was strictly reserved for the synthesis of empirical telemetry, the generation of visualizations, code packaging for reproducibility, and the completion of this final dissertation manuscript. Complex procedural terrain generation was formally delegated to future work.

1.4 Dissertation Outline

This dissertation is structured to reflect the continuous engineering pipeline of the project, beginning with the theoretical foundations of deep reinforcement learning, legged biomechanics, and the sim-to-real gap established in Chapter 2 (Background). Building upon this foundational literature, Chapter 3 (Model Architecture) details the methodological implementation of the digital twin, including the mechanical CAD-to-URDF pipeline, the formalization of the Markov Decision Process (MDP), and the specific reward structures and domain randomization techniques applied within the NVIDIA Isaac Lab environment. The efficacy of this control architecture is then empirically evaluated in Chapter 4 (Experimental Results), which presents a rigorous quantitative analysis of algorithm convergence, kinematic tracking accuracy, and hardware safety compliance, supplemented by time-series telemetry of the robot's dynamic disturbance rejection. Finally, Chapter 5 (Conclusions) summarizes the core research outcomes, contextualizes the project's broader engineering impact, and outlines concrete pathways for future physical hardware calibration and exteroceptive integration.

2. BACKGROUND

2.1 Key Concepts

2.1.1 Reinforcement Learning and PPO

Reinforcement Learning (RL) is a subfield of machine learning uniquely suited for sequential decision-making and continuous control in robotics. Unlike supervised learning, which relies on static, pre-labelled datasets, RL operates on a paradigm of exploration and exploitation. An artificial agent learns to navigate its environment by executing actions and receiving numerical feedback in the form of rewards or penalties. Over time, the agent optimizes its behaviour to maximize the cumulative reward it receives (Sutton and Barto, 2018).

Mathematically, this interaction is formalized as a Markov Decision Process (MDP), typically defined by the tuple $\langle S, A, P, R, \gamma \rangle$. In the context of quadrupedal locomotion, S represents the continuous state space comprising the robot's sensor readings (such as joint positions, velocities, and chassis orientation), and A represents the continuous action space (such as target motor torques or position commands). The transition dynamics, $P(s_{t+1} | s_t, a_t)$, are governed by the physics engine of the simulation. At each control step t , the agent receives a scalar reward $r_{t+1} = R(s_t, a_t)$ based on a shaped reward function designed to encourage desired behaviours, such as matching a target velocity, while penalizing instability or excessive energy consumption. The discount factor $\gamma \in [0,1)$ dictates the agent's foresight, balancing the importance of immediate stabilization against long-term gait sustainability.

The core objective of the RL algorithm is to learn an optimal policy, $\pi_\theta(a | s)$, parameterized by a neural network with weights θ , that maps states to the most advantageous

actions. For highly complex, high-dimensional robotics tasks, Proximal Policy Optimization (PPO) has emerged as the industry standard (Schulman et al., 2017). PPO utilizes a dual-network architecture known as "actor-critic", where one neural network (the actor) determines the robot's physical movements, while a second network (the critic) evaluates the success of those movements. Furthermore, it is an "on-policy" algorithm, meaning it learns and adapts exclusively based on its current interactions with the environment rather than relying on outdated past data. Unlike older algorithms that discard simulation data after a single evaluation, PPO achieves high sample efficiency by retaining and reusing batches of collected experiences for multiple incremental updates. Its defining mathematical feature is a mechanism that strictly limits how much the policy can change during a single training step. By preventing the neural network from altering its behaviour too drastically at any one time, PPO successfully avoids the catastrophic performance collapse—or sudden "forgetting"—that frequently plagues other algorithms when learning continuous walking motions.

2.1.2 Legged Locomotion and Biomechanics

Legged locomotion relies on periodic, repeatable sequences of foot placements known as gaits. The primary objective during these gaits is to translate the body forward while maintaining the stability of the system's Center of Mass (CoM), despite the inherent vertical and lateral displacements the body experiences during the transition between stance and swing phases. In biological quadrupeds, such as canines and equines, gaits are naturally selected to optimize energetic efficiency and manage this balance at varying speeds (Alexander, 2002).

Slower locomotion is typically achieved through a 'walk', a four-beat sequence where the limbs move sequentially. In a walk, at least three feet remain in contact with the ground at all times, creating a constant two-dimensional base of support (a triangle). This maximizes static stability, meaning the system's CoM remains safely within its physical footprint, allowing it to theoretically pause mid-stride without tipping over.

In contrast, a 'trot' is a faster, two-beat gait where diagonal pairs of legs (for example, the front-left and rear-right) swing and strike the ground simultaneously. During a trot, only two feet contact the ground at any given moment. Because two points of contact form a one-dimensional diagonal line rather than a two-dimensional base, static balance is mathematically impossible; gravity immediately begins pulling the CoM sideways. To prevent collapsing, the system relies on dynamic stability. Instead of balancing perfectly, the robot uses its forward momentum and the rapid, precisely timed placement of the opposite diagonal pair of feet to intercept the ground, continuously "catching" its falling weight and transferring its momentum from one unstable step to the next.

2.1.3 The Sim-to-Real Gap

In reinforcement learning for robotic locomotion, the most critical bottleneck is the Sim-to-Real gap—the inevitable divergence between a mathematically perfect simulation and the chaotic, physical reality of the hardware. Simulators naturally calculate dynamics with perfect sensory feedback, zero communication latency, and virtual actuators capable of instantaneous, infinite torque. When a policy trained under these idealized conditions is deployed to a physical

robot, the accumulation of real-world mechanical realities rapidly compounds, causing the policy to fail and the system to collapse.

To mitigate these catastrophic failures, modern locomotion frameworks rely on the development of a transfer-ready digital twin. As established in the introduction, treating the simulator not just as a theoretical training ground but as a strict diagnostic sandbox allows learned policies to be stress-tested against physical constraints long before a prototype is deployed.

Building this transfer-ready environment requires intentionally degrading the perfect simulation to mirror real-world imperfections. While some methodologies rely on injecting physical chaos (Peng et al., 2018) alongside complex terrain generation (Lee et al., 2020), doing both simultaneously during early development is often premature and destabilizes baseline gait synthesis. Therefore, the digital twin in this study employs a targeted isolation strategy. The topographical complexity is deliberately simplified to a flat plane, allowing the policy to isolate and master the foundational biomechanics of the trot gait. However, to ensure this baseline policy is hardware-ready and capable of bridging the Sim-to-Real gap, comprehensive domain randomization is injected directly into this flat environment. By subjecting the robot to unpredictable payload masses, dynamic Center of Mass (CoM) shifts, varying floor and joint friction, actuator gain degradation, simulated sensor noise, external physical perturbations, and strict actuator limits on a flat surface, the system forges a highly robust locomotion controller without the compounding failure rates of rough terrain training.

2.2 Literature Review

2.2.1 Introduction

This dissertation investigates reinforcement learning (RL) for rigid quadruped locomotion within a simulation-based “digital twin” workflow. The motivating context is locomotion in rough, cluttered, and hazardous or hard-to-access environments where terrain discontinuities, debris, uncertain friction, disturbances, and sensing degradation undermine conventional mobility assumptions. The aim is not to produce a gait in benign conditions, but to obtain robust locomotion: stable and repeatable behaviour under uncertainty, with explicit evidence of degradation modes and recovery mechanisms.

Importantly, this project is locomotion-only and simulation-only: the physical prototype is not available within the dissertation timeframe, so real-world deployment and sim-to-real validation are out of scope. Nonetheless, the digital twin should be built “transfer-ready by design” (i.e., with hardware-aligned interfaces, calibration parameters, and robustness evaluation) so that future work can calibrate and deploy it once the prototype exists. In this review, “digital twin” therefore refers to the simulation model plus the associated training/evaluation toolchain and experiment configurations, rather than an onboard component.

Across recent legged-robot literature, deep RL has repeatedly demonstrated strong simulated performance and, in many studies, successful transfer to hardware. At the same time, the reality gap (contact behaviour, actuator dynamics, delays, estimator drift, and sensor noise) is a recurring reason policies fail outside training conditions (Bellegarda et al., 2021; Lee et al., 2019; Tang et al., 2024; Plozza et al., 2025; Gurram, 2024; Bjelonic et al., 2025). For this

dissertation, the practical takeaway is that robustness mechanisms are worth adopting even before hardware exists, because they reduce overfitting to a single simulator setting.

2.2.2 Digital Twin Workflow for Locomotion

2.2.2.1 Simulation framework and reproducibility

GPU-accelerated simulators have enabled massively parallel RL training and systematic robustness testing. Isaac Lab provides an end-to-end simulation and learning framework with configurable actuator models, multi-rate sensing, terrain generation, domain randomization, and standardized environments that support locomotion research and reproducible experiment configurations (Isaac Lab Team, 2025).

A useful digital twin is more than a simulator: it is a reproducible experiment setup. In practice this requires versioned robot assets (URDF and meshes), explicit definitions of observations and actions, fixed seeds for benchmark runs, and a standard evaluation suite so policies can be compared fairly across training variants. Digital twin “synchronization” studies in other robotics domains also illustrate how simulation can be coupled to real-time middleware and latency-aware pipelines when hardware exists (Ali et al., 2025); this supports framing the digital twin as an engineering asset rather than a purely academic simulator.

2.2.2.2 Reality-gap awareness without claiming deployment

Even when real deployment is deferred, reality-gap literature is still useful because it explains why controllers that look strong in simulation can degrade when key assumptions change. Aljalbout et al. (2025) formalise the notion of a “gap” between simulated and real performance

and organise common causes into categories such as dynamics/contact effects, sensing/estimation errors, and actuation/control limitations. This framing is valuable for locomotion because many failure modes are tied to contact modelling and timing, actuator limits, and imperfect state information rather than to policy architecture alone. (Aljalbout et al., 2025)

For a locomotion-only dissertation, the implication is methodological rather than claim-based: robustness should be argued using performance across planned domain shifts (e.g., friction variation, actuation delay, actuator limits) rather than a single nominal simulator score. The same guidance also motivates action regularisation (penalising overly large or rapidly changing actions) because it discourages high-frequency “bang–bang” behaviours that can exploit simulator artefacts and become brittle under small mismatches. (Aljalbout et al., 2025)

2.2.3 Foundations: Balance, Terrain Interaction, and Robustness

2.2.3.1 Foot placement and simple balance templates

Classical locomotion and balance research provides compact models and interpretable robustness mechanisms that remain useful as conceptual anchors. Linear Foot Placement Control (LFPC) shows that simple velocity-feedback foot placement rules can stabilize dynamic walking, and highlights how step timing affects stability margins (Ye et al., 2022).

Template models such as SLIP also show how robustness can be decomposed into touchdown control, attitude regulation, and energy regulation. A 3D-SLIP strategy demonstrates recovery from sudden external impulses (pushes/collisions) using a structured controller (touchdown

angle, attitude regulation, and energy compensation), and reports quantitative disturbance thresholds that can guide robustness tests. (Zhang et al., 2022)

2.2.3.2 Compliant terrain and contact uncertainty

Real terrain is not always perfectly rigid; compliance (how much a surface deforms under load) changes contact forces and the timing of stance. Karakasis et al. (2022) model compliant ground contact and show that adapting the leg’s effective stiffness in real time during walking maintains stability under sudden changes in ground stiffness better than keeping stiffness fixed. This suggests that the locomotion policy should be evaluated under contact variability, in particular friction changes and terrain-induced timing disturbances, since real foot–ground interaction is uncertain, even if the simulator approximates the ground as rigid.

2.2.3.3 Recovery and push robustness beyond steady walking

Robust locomotion is not only steady walking on a clean surface; it also includes recovering from disturbances such as pushes, slips, or sudden terrain changes. Chen et al. (2022) provide a model-based way to define when a quadruped is still “recoverable” after a disturbance (i.e., whether balance can be regained through future steps). This provides a useful conceptual basis for designing a simulation evaluation suite that explicitly tests push recovery, low-friction slip events, and step/discontinuity handling, rather than reporting performance only on nominal flat-ground tracking. (Chen et al., 2022)

2.2.4 Deep RL for Quadruped Locomotion: Problem Formulation and Control Interfaces

2.2.4.1 Deployable action interfaces and low-level control

Recent rigid-quadruped RL studies typically embed the learned policy within a broader control architecture. Instead of commanding raw torques directly, policies often output intermediate targets (e.g., joint position offsets or desired foot positions relative to the robot body) that are tracked by a low-level controller, improving stability and making the learned policy less sensitive to actuator limits, delays, and modeling errors. (Bellegarda et al., 2021; Zhang et al., 2024).

Zhang et al. (2024) present Ask1, a custom quadruped platform, where the learned policy outputs 12 joint-position offsets that are tracked by a PD controller. Using PD-tracked targets constrains how extreme the policy’s commands can be and typically reduces sensitivity to small modelling errors and control latency, which is helpful if the controller is later evaluated on hardware. Bellegarda et al. (2021) similarly show that the action representation, for example, commanding joint targets versus commanding desired foot placements, can strongly affect learning stability and robustness, so it should be treated as a deliberate design choice rather than a minor implementation detail.

2.2.4.2 Structural decompositions and coordination

Some locomotion work builds in structure by splitting the control problem across legs rather than learning a single monolithic controller. For example, MASQ trains a cooperative multi-agent policy where each leg is treated as a separate agent (MAPPO), and reports results on a Unitree Go2 platform (Liu et al., 2024). Even if this dissertation uses a single policy, MASQ is

useful as evidence that “controller factorisation” (per-leg vs whole-body) is a design choice that can affect training stability (whether learning improves consistently without sudden collapse or erratic behaviour during training) and robustness. (Liu et al., 2024)

2.2.4.3 Surveys and field-level perspectives

Two recent surveys synthesize the rapid expansion of RL locomotion, including common training practices (randomization, curricula, privileged learning) and typical failure modes (contact mismatch, actuator dynamics, latency) (Gurram, 2024; Yue, 2023). A broader survey of real-world DRL successes in robotics further emphasizes that robust deployment typically relies on multiple overlapping mechanisms rather than a single “magic” technique (Tang et al., 2024).

Mirza and Singh (2025) argue that imitation learning can reduce trial-and-error and stabilise early locomotion training by learning from example trajectories. They caution that demonstrations generated under different robot dynamics or contact conditions can mislead the policy and still require RL fine-tuning.

2.2.5 Robustness in Simulation: Domain Shifts, Randomization, and What to Model

2.2.5.1 Domain and dynamics randomization as robustness

Domain randomization differs substantially across locomotion studies in what is varied and how aggressively. Common choices include contact parameters (friction), inertial properties (mass/CoM scaling), actuation characteristics (strength/limits and delay), sensing noise, and terrain generation; these choices shape not only final performance but also learning stability and failure modes (Bellegarda et al., 2021; Li et al., 2021; Yu et al., 2022; Zhang et al., 2024; Roux

et al., 2025; Plozza et al., 2025). A recurring trade-off is that narrow randomization can yield brittle, simulator-specific gaits, while overly broad randomization can slow learning or push policies toward overly cautious gaits (e.g., slower motion and smaller steps) that avoid failure rather than achieving agile locomotion. For this reason, both the randomization ranges and the difficulty schedule used during training (starting easy and progressively increasing terrain difficulty and uncertainty) are deliberate design choices and should be reported explicitly in a simulation-only study.

Selecting randomization ranges is not straightforward: ranges that are too narrow can produce policies that work only in a single simulator setting, while ranges that are too wide can slow learning or encourage overly cautious gaits. DROPO (Domain Randomization Off-Policy Optimization) is a data-driven method that tunes the domain-randomization distribution using previously logged rollouts (recorded trajectories), typically from the target robot, rather than choosing ranges purely by hand. Because the physical prototype is not available within this dissertation, such data-driven tuning is out of scope; however, DROPO provides a clear future calibration pathway and motivates explicit reporting of randomization ranges in the present simulation study. (Tiboni et al., 2023)

2.2.5.2 Friction, delays, and actuator mismatch

Several sources highlight that friction and delays can dominate failure modes. A recent sim2real analysis shows that excluding static friction from randomization can cause policies to succeed in simulation yet fail in real walking, and provides friction-aware randomization examples (Hu et al., 2025).

Actuation and timing effects are recurring issues in legged locomotion. In work on the Cassie biped robot, researchers model uncertainty by randomising communication/control delays (among other dynamics perturbations) and show that these choices influence robustness (Li et al., 2021; Yu et al., 2022). A constrained-RL study on the Bolt point-foot biped similarly identifies actuation delay and joint friction as critical factors, and stabilises training using “constraint-as-termination” (ending an episode when safety limits are violated) (Roux et al., 2025). Masuda and Takahashi (2023) show that, in gear-driven robots, transmission effects such as friction and delay can be the main source of mismatch between simulation and reality. They therefore argue for explicit actuator modelling and for re-tuning those actuator parameters using measured data when available.

2.2.5.3 Constrained/hybrid robustness mechanisms

Some methods increase robustness by combining reinforcement learning with additional stabilising control layers. RL2AC (“Reinforcement Learning–based Rapid Online Adaptive Control”) keeps an RL locomotion policy but adds a very fast correction loop that adjusts joint torques during walking to counter disturbances and modelling errors (Zhou et al., 2024). Other work combines RL with model predictive control, a controller that computes actions using an explicit physics/dynamics model of the robot, to plan a short time ahead while enforcing constraints (e.g., balance and torque limits). In this hybrid setup, model predictive control (MPC) handles immediate stability, while the learned component provides extra guidance. This helps MPC’s short-horizon plan remain well-behaved, leading to more reliable locomotion overall (Zhou et al., 2023).

2.2.6 Partial Observability, Memory, and Estimation

2.2.6.1 POMDP framing and observation histories

In practice, legged locomotion is partially observable: sensors are noisy and delayed, foot contact can switch unpredictably, and state estimates can drift, so the controller never has perfect access to the true state at each instant (a POMDP, i.e., a Partially Observable Markov Decision Process). DreamWaQ—an RL locomotion method proposed by Nahrendra et al. (2023)—addresses this by giving the policy a short history of proprioceptive (internal) signals (joint measurements and IMU signals). Using this recent history, the policy learns an internal “memory” that serves as a best-guess of ground effects (for example whether the robot is on a slope, starting to slip, or stepping on a softer surface that deforms under load), without using external environment sensors such as cameras or LiDAR. Plozza et al. (2025) make a related point by treating state-estimation errors as part of the problem: during training they inject both random noise and systematic offsets/drift into the observation/estimation pipeline, so the controller remains stable even when the state estimate is imperfect.

Because locomotion is partially observable, many studies use policies with memory—either policies with built-in memory (e.g., LSTM/GRU) or policies fed explicit history features—so decisions can depend on recent sensor context rather than only the current timestep. Song et al. (2018) demonstrate this idea on rough-terrain biped locomotion, showing that memory-based policies can handle hidden dynamics that are not directly observable from instantaneous measurements. More recent work on partial-observation humanoid control similarly reports that

removing history information can cause training to fail, while explicit history encoding can restore robust performance (Wang & Chen, 2025).

2.2.6.2 Curriculum learning and long-context adaptation

Curriculum learning (i.e., a training difficulty schedule where tasks start easy and become progressively harder) is commonly used in locomotion RL to improve training stability and robustness. HACL (History-Aware Curriculum Learning) makes this idea explicit for locomotion by using recent history to guide how difficulty is increased (e.g., terrain variation, command ranges, disturbances), aiming for faster and more robust policies (Mishra et al., 2025). Beyond short history windows, LocoFormer (Liu et al., 2025), a locomotion policy that does not rely only on the robot’s current sensor reading, uses a model that can look back over a long window of recent internal sensor readings and automatically decide which moments in that window matter most for choosing the next action. This helps the policy adjust its gait when conditions change (for example, when the ground becomes slippery or uneven). While that scale is beyond MSc scope, it supports a focused takeaway relevant here: robustness is influenced both by how training difficulty is staged (curriculum) and by how much recent context the policy can use (long-context adaptation).

2.2.6.3 Estimation and privileged information during training

Some studies make training easier by giving the learning system privileged information—extra simulator-only signals that would not be directly available from the real robot’s onboard sensors at runtime. For example, in simulation you can know the exact terrain height at each foot or the exact external force applied during a push. To avoid “cheating” at deployment, these extra

signals are used only during training to guide learning, while the final controller is trained to operate using only realistic onboard sensors (e.g., joint encoders and an IMU). DreamWaQ and Ask1 follow this approach by using privileged terrain/disturbance information during training but keeping the deployable policy based on realistic observations (Nahrendra et al., 2023; Zhang et al., 2024). APRiL (Attention-Privileged Reinforcement Learning) applies a similar idea in vision-heavy settings by using an auxiliary privileged learner to help a sensor-limited policy train reliably under strong visual variations (Sikchi et al., 2020).

Some papers try to identify which hidden quantities the controller really needs for robust walking; Wang et al. (2024) study this by training a locomotion policy together with an auxiliary estimator and then analysing which estimated signals most affect performance; in their experiments, accurate body velocity estimation emerges as especially important for robustness. Nguyen et al. (2025) take a more general view and argue that “privileged” training signals do not have to be full state: any extra simulator-only signal can be useful during training, and they propose criteria for judging whether a candidate training-only signal is informative.

Some studies use teacher–student training, where a stronger “teacher” controller is trained first and then a “student” controller is trained using the teacher as guidance. Learn to Teach (Wu et al., 2025) is one such method designed to make locomotion training more sample-efficient on uneven terrain. Distillation-PPO (Zhang et al., 2025a) combines two ideas: (1) the student is trained to imitate the teacher (so it learns stable behaviour quickly), and (2) the student is then further improved using PPO (Proximal Policy Optimization), a standard RL algorithm that updates the policy in small, stable steps. During this improvement stage, an imitation term is

kept as a regularising constraint that discourages the student from drifting too far from the teacher, helping maintain stability while PPO continues to improve performance.

2.2.7 Robustness Beyond Nominal Walking: Skills, Recovery, and Composition

Robust locomotion is not only steady walking; it also includes recovery from failure states and switching between basic skills. On the ANYmal quadruped platform, a learned recovery system combines multiple skills—such as self-righting, standing up, and normal locomotion—using a simple behaviour selector to choose which skill to run. Lee et al. (2019) analyse failure cases and identify degraded state estimation as a major cause of failure, underscoring that robustness depends on the sensing–control loop, not only the policy architecture.

Other work targets rough-terrain agility and specialised skills. Kim et al. (2023) train terrain-adaptive locomotion by imitating animal motions and add a separate “terrain adaptation” component that produces a bounded, state-dependent correction added to the base action (an action offset), improving robustness across challenging terrains. ZSL-RPPO (“Zero-Shot Learning with Recurrent PPO”) trains a PPO-based locomotion policy with built-in memory (e.g., an LSTM/GRU) for difficult terrains and reports robustness across multiple terrain types; it also explicitly contrasts robustness achieved via simulation randomization with approaches that rely on hardware-based system identification (Zhao et al., 2024).

Angarola et al. (2025) suggest that using several policies, each tuned for a different terrain type, can outperform one “do-everything” policy on some challenging terrains—provided there

is a way to choose the appropriate policy at runtime (e.g., a simple terrain classifier or gating rule).

Controller structure can itself improve robustness. Duan et al. (2025) argue that layered architectures—with a fast low-level stabilising controller and slower higher-level components—can produce more robust locomotion than a single monolithic policy. They also suggest training a strong stabilising “base walker” first and then adding complexity later, which motivates careful choice of what the policy controls (e.g., torques vs joint targets vs foot-placement targets) and of the training schedule.

Recent work also treats terrain generation as a robustness lever. GenTe (“Generative Real-world Terrains”) is a framework that builds a large library of automatically generated rough terrains in simulation and uses them for curriculum training and evaluation; it reports better generalisation when policies are trained and tested on many diverse terrains rather than a small hand-designed set (Wan et al., 2025).

While these advanced topological methodologies represent the future of deployment, establishing a mechanically robust, parallel-stance baseline on planar terrain remains the mandatory prerequisite prior to introducing such spatial complexity.

2.2.8 Alternatives and Complements to Vanilla PPO

2.2.8.1 Hybrid model-based and learning-based approaches

Yang et al. (2022) tune parameters of a Hybrid Zero Dynamics (HZD) walking controller on the Cassie biped, using Bayesian optimization, which is designed to find good parameters with relatively few trials while enforcing safety constraints to reduce the risk of falls. In a related direction, Nguyen et al. (2025b) extend HZD by using a precomputed gait library as guidance and then applying PPO to adapt behaviour during execution, illustrating how structured model-based gaits and RL can be combined.

A benchmarking study in the MuJoCo physics simulator compares model predictive control (MPC) and reinforcement learning (RL) controllers on the Unitree Go1 quadruped and reports different strengths: RL tolerates larger front-back direction disturbances and achieves better energy efficiency (lower cost of transport, CoT), while MPC performs better for certain sideways disturbances (Patel et al., 2025). This motivates evaluating and reporting robustness using multiple metrics (e.g., disturbance tolerance in different directions and energy efficiency) rather than relying on a single reward value.

2.2.8.2 On-policy vs off-policy comparisons and optimization stability

Algorithm choice is often justified by training stability and by the robustness of the resulting gait. Mock and Muknahallipatna (2023) compare PPO with two alternative RL algorithms, TD3 and SAC, for quadruped gait generation, and report that results depend strongly on what sensor inputs are provided to the policy and how many input variables are used.

Beyond RL trained purely from reward signals, some studies make locomotion training more reliable by giving the learner extra help at the start. As concrete examples, Bogdanovic et al. (2022) use demonstration trajectories generated by trajectory optimisation (precomputed example motions) so the policy begins from a reasonable gait instead of random actions, while Beranek et al. (2022) improve disturbance handling by switching between a small library of predefined behaviours rather than relying on a single controller.

Even with a strong base algorithm and initial demonstrations, keeping the training mathematically stable as it runs is a major challenge. Standard methods try to prevent the neural network from changing too drastically in a single step, but if the immediate previous update was a poor one, the learning process can still be pulled in the wrong direction. To solve this, Le et al. (2022) have proposed the Memory-Constrained Policy Optimization algorithm (MCPO). Instead of anchoring the policy's updates to just one previous step, MCPO keeps a memory buffer of older, successful policies to form a "virtual trust region". By mathematically constraining new updates to stay close to this historical average, the algorithm prevents sudden learning collapses—where the system effectively "forgets" previously learned behaviours—and thus demonstrates improvements in various benchmarks, including locomotion tasks.

Another approach to stability is enforcing strict physical boundaries. In standard RL, policies can sometimes learn erratic motions while chasing higher rewards. To counter this, constrained reinforcement learning explicitly restricts unsafe states. In their work on the Bolt robot, Roux et al. (2025) enforce these boundaries using a "constraints-as-terminations" approach. By instantly ending a training simulation the millisecond the robot violates a safety limit—such as an unsafe

joint speed—the system forces the controller to avoid destructive exploration. Finally, some researchers expand robustness by actively attacking the policy. Zhang et al. (2025b) employ an adversarial training regime; rather than applying random noise, the simulation calculates and applies the specific, targeted physical disturbances most likely to make the robot fail. Forcing the controller to survive these worst-case attacks while imitating complex motions results in a highly resilient recovery policy.

2.2.9 Synthesis and Dissertation Gap

A core consensus across the reviewed literature is that robust legged locomotion is achieved through overlapping mechanisms rather than a single algorithmic solution. Successful architectures routinely combine structured action interfaces and low-level control (Bellegarda et al., 2021; Zhang et al., 2024), domain shift training and evaluation (Bellegarda et al., 2021; Li et al., 2021; Hu et al., 2025; Tiboni et al., 2023), memory and estimation to handle POMDP settings (Nahrendra et al., 2023; Plozza et al., 2025; Song et al., 2018; Liu et al., 2025), privileged learning pipelines (Sikchi et al., 2020; Wu et al., 2025; Nguyen et al., 2025), and hybrid constraint mechanisms (Yang et al., 2022; Zhou et al., 2024; Roux et al., 2025). Furthermore, recent survey literature confirms that translating these simulated successes to hardware requires explicit consideration of actuation dynamics, sensing limitations, and rigorous evaluation methodologies (Tang et al., 2024; Gurram, 2024; Yue, 2023; Bjelonic et al., 2025).

Within an MSc scope, the primary research gap is investigating how a simulation-based digital twin can be used to engineer feasible locomotion for a custom hardware platform. Accordingly, this dissertation positions its contribution as: (i) constructing a rigid quadruped

digital twin derived from the target CAD-to-URDF chain inside Isaac Lab, (ii) implementing a PPO-based locomotion controller exploring the reward shaping required to force feasible flat-ground locomotion and (iii) documenting gait benchmarks to establish realistic performance baselines and understand the mechanical constraints of the physical prototype.

3. MODEL ARCHITECTURE

3.1 Model Description

The system architecture developed for this project relies on a continuous feedback loop between physical hardware design, rigorous physics simulation, and advanced control algorithms. The architecture is divided into three primary components that together form the complete digital twin and Reinforcement Learning (RL) pipeline.

The foundational component of the system is the physical design of a custom quadrupedal robot. Designed for mobility in unstructured and hazardous environments, such as nuclear decommissioning zones, the robot has a total system mass of 64.36 kg and operational dimensions of 1.14 m in length, 0.81 m in width, and standing height of 0.94 m . The mechanical structure consists of a central chassis (the base link) and four highly articulated legs, each featuring three degrees of freedom (hip, thigh, and calf joints). The initial design was constructed using Computer-Aided Design (CAD) software (*SolidWorks*), which established the precise geometric bounds, Center of Mass (CoM), and inertia tensors required for accurate real-world representation.

To safely train the locomotion controller without risking catastrophic damage to the physical hardware, the mechanical design was mathematically translated into a nominal digital twin. First, the CAD assembly was exported into a Unified Robot Description Format (URDF). This intermediate file was critical for establishing the robot's kinematic tree, defining the precise joint limits, and mapping the foundational inertial properties (mass and Center of Mass) of each individual link. During this formulation, the physical rubber contact pad at the distal end of each

leg was modeled as a discrete, independent rigid body (the "foot" link) attached to the calf via a fixed, zero-degree-of-freedom joint. While firmly integrated in physical reality, isolating this foot mesh within the URDF is a necessary mechanical abstraction; it provides the downstream physics engine with the discrete collision geometries required to mathematically distinguish the bottom of the foot from the rest of the leg structure. The individual structural links defined during this CAD-to-URDF pipeline are illustrated in Figures 2 through 5, while their corresponding kinematic parameters and mass distributions are summarized in Table 1.

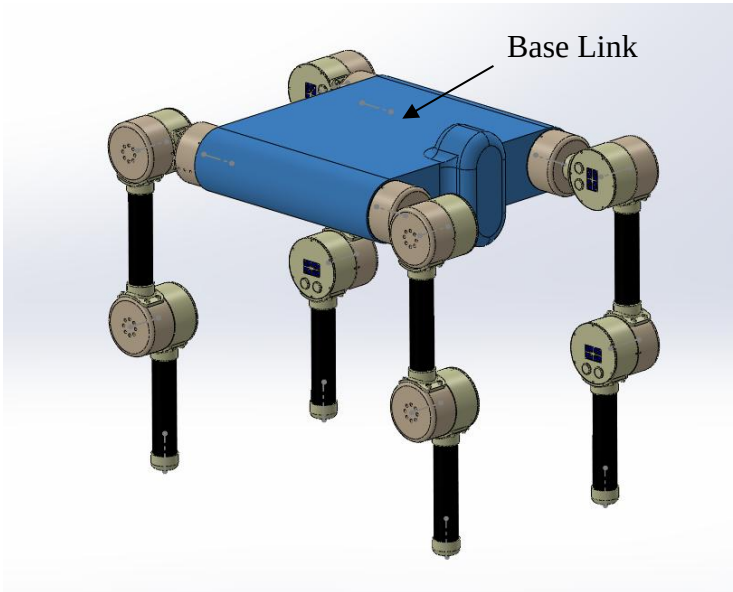


Figure 2: The Base Link

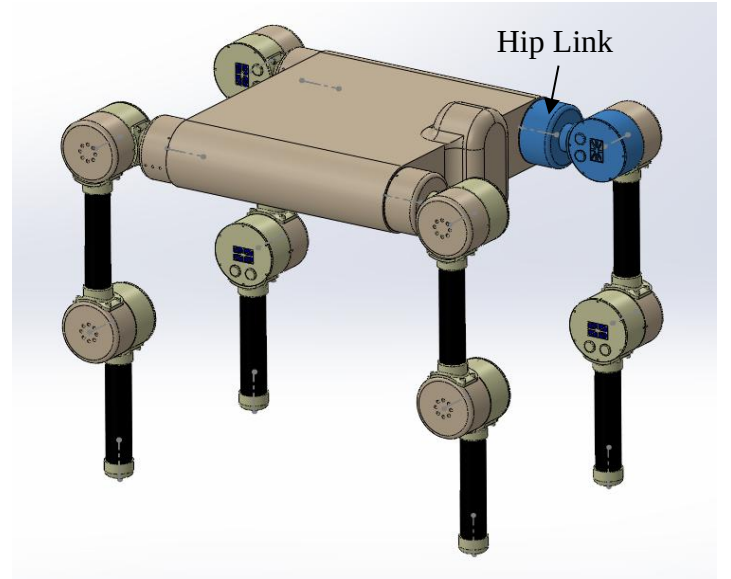


Figure 3: A Hip Link

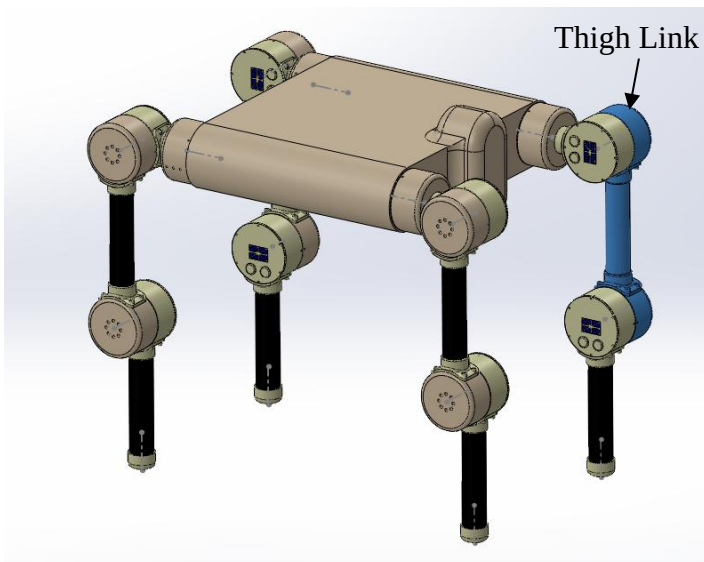


Figure 4: A Thigh Link

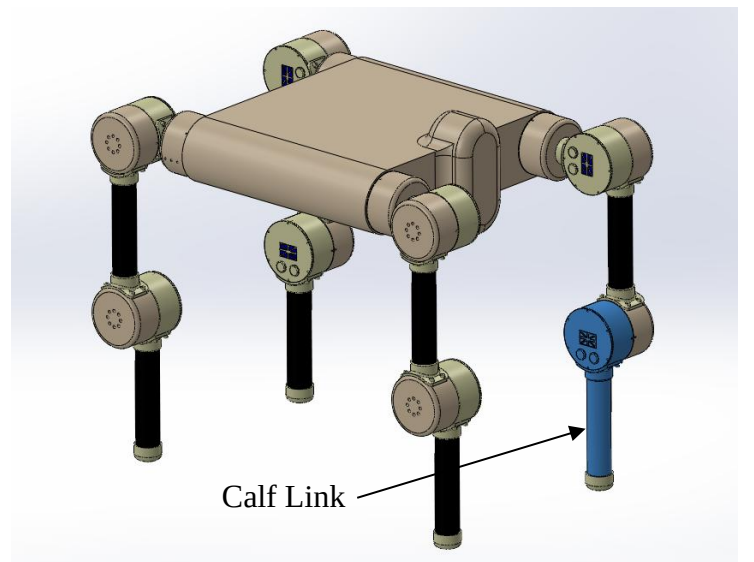


Figure 5: A Calf Link

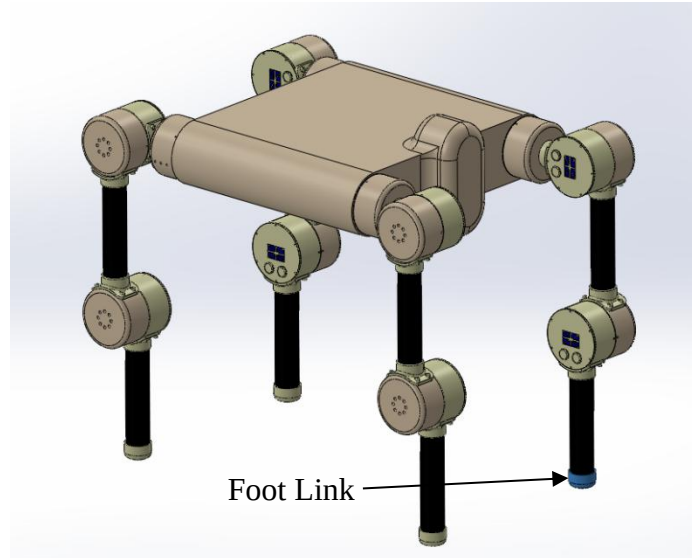


Figure 6: A Foot Link

Table 1: Kinematic and inertial properties of the quadruped robot

Component Assembly	Mass per Link (kg)	Rotation range (°)	Nominal Starting Angle(°)
Base Chassis (Base Link)	14	-	-
Hip (All Legs)	5	-25.8 to 25.8	0
Thigh (Front-Left, Rear-Right)	5	-30 to 0	-11.5
Thigh (Front-Right, Rear-Left)	5	0 to 30	11.5
Calf (Front-Left)	2.5	-30.0 to 30.0	-22.9
Calf (Rear-Right)	2.5	0.0 to 30.0	22.9
Calf (Front-Right)	2.5	-30.0 to 30.0	22.9
Calf (Rear-Left)	2.5	-30.0 to 0.0	-22.9
Foot Pad (All Legs)	0.09	-	-

Following this baseline formulation, the URDF was imported and converted into a Universal Scene Description (USD) asset, which serves as the core digital twin hosted within the NVIDIA Isaac Lab simulation environment. Powered by the underlying PhysX engine, the simulator strictly enforces the robot's rigid body collisions, gravitational forces, and the kinematic boundaries defined in the URDF. While this meticulously defined USD asset provides the precise geometric and inertial baseline, the simulation pipeline is designed to intentionally degrade these perfect virtual conditions; it enforces strict maximum actuator torques and subsequently injects bounded domain randomization (as rationalized in Section 2.1.3) to simulate the unpredictable mechanical degradation of the physical world. The fully rendered USD digital twin within this environment is visualized in Figure 7.

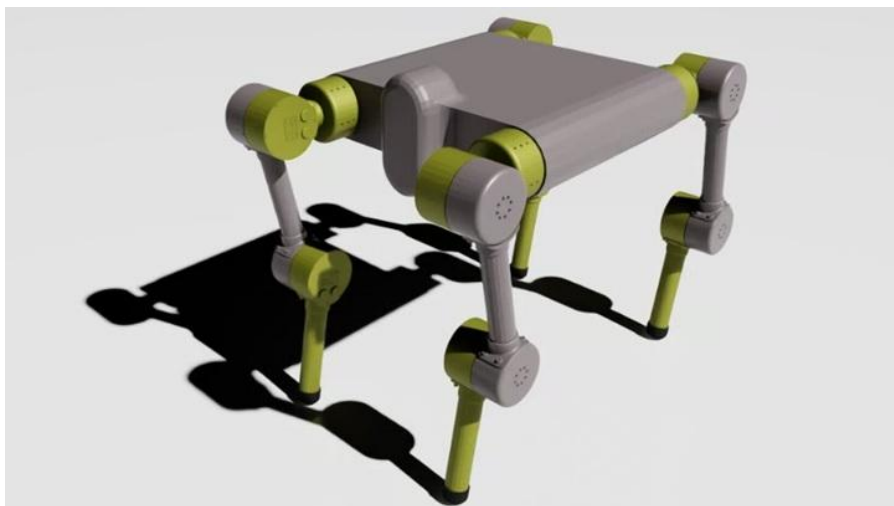


Figure 7: The quadruped rendered as a USD asset within NVIDIA Isaac Lab

The final component of the architecture is the control logic that drives the physical model. The robot is governed by a Proximal Policy Optimization (PPO) reinforcement learning algorithm trained entirely from scratch. Rather than relying on pre-programmed, hand-crafted

walking trajectories, this artificial agent acts as the system's "brain." It processes real-time continuous states from the simulation, compiling sensor data—such as joint positions, velocities, and chassis orientation—into a 235-dimensional numerical array known as the observation vector (the exact composition of these 235 dimensions is detailed in Section 3.2.3). The neural network processes this vector and outputs 12 synchronous target positions to the robot's leg joints. These high-level neural commands are then translated into physical movement via low-level Proportional-Derivative (PD) controllers, which drive the simulated actuators to propel the quadruped forward.

3.2 Model Analysis

3.2.1 Actuator Dynamics and Joint Control

The foundation of the digital twin's physical realism lies in its actuator dynamics. In the NVIDIA Isaac Lab environment, the robot's joints are not animated kinematically; instead, they are driven by simulated motors that enforce strict physical constraints. The translation between the high-level neural network commands and the low-level physical movement is governed by a Proportional-Derivative (PD) controller at each of the 12 joints.

The reinforcement learning algorithm outputs a target position (θ_{target}) for each joint. The PD controller continuously compares this target position against the joint's current physical position ($\theta_{current}$) and current velocity ($\dot{\theta}_{current}$) to calculate the precise torque (τ) required to move the limb. This closed-loop control system is mathematically defined by the following equation:

$$\tau = K_p(\theta_{target} - \theta_{current}) - K_d\dot{\theta}_{current}$$

Wherein:

- τ - the resulting torque applied to the physical joint.
- K_p - the proportional gain (stiffness). The nominal baseline is calibrated to 800.0. While standard pipelines often begin with a static analytical derivation to estimate required holding torques, such theoretical models are insufficient in simulation; they fail to account for the non-linear ground contact dynamics and the discrete time-stepping constraints inherent to the Isaac Lab physics engine. In a continuous physical system, restorative forces adjust instantaneously. However, in a discrete simulation, physics are resolved in fixed time steps. If the proportional gain is set too high, microscopic positional errors upon ground impact generate massive torque commands. Because the simulator holds this torque constant across the entire time step, the joint violently overshoots its target. This creates an amplifying mathematical feedback loop—a numerical instability—that ultimately causes the collision solver to forcefully eject the robot from the environment. Therefore, the final magnitude was established through several dozen empirical simulation sweeps. 800.0 was identified as a highly effective operational baseline: when paired with appropriate damping, it provides sufficient restorative force to prevent the 64.36 *kg* chassis from collapsing under gravity, while remaining safely below the upper bounds where numerical instability occurs. Notably, this value aligns perfectly with established Isaac Lab defaults for heavy-duty industrial manipulators, validating the empirical findings against the quadruped's actual high-torque actuator architecture. This value is subject to $\pm 15\%$ scaling during the Domain Randomization phase to simulate mechanical wear.

- K_d - the derivative gain (damping), which acts as a viscous braking force. The nominal baseline is calibrated to 50.0. Because the high stiffness ($K_p = 800.0$) required to support the chassis naturally introduces severe joint oscillation upon impact, this damping parameter was established in tandem through empirical trial-and-error tuning. During these sweeps, 50.0 was identified as the optimal baseline to act as a shock absorber; it provides sufficient viscous friction to damp the oscillations without rendering the joints too sluggish. Just as with the stiffness, this damping value directly mirrors the standard default configurations for heavy robotic arms, confirming that the empirically selected parameters appropriately match the physical realities of the high-inertia limbs. This value is similarly subjected to $\pm 15\%$ scaling during the Domain Randomization phase. (Note: Based on manufacturer specifications, the published static backdriving torque (stiction) for the physical Harmonic Drive CSG-25 gearboxes is 14 *Nm*. However, implementing this true static limit within the rigid-body simulation induces kinematic instability and prevents the policy from synthesizing a viable gait. To enable successful training convergence, the simulated joint friction was empirically reduced to a 3.5 *Nm* baseline. Crucially, this sim-to-real discrepancy does not compromise physical deployment. Because the reinforcement learning policy outputs target joint positions rather than direct torque commands, the physical robot relies on its high-frequency, low-level PD controllers. These hardware controllers naturally draw the transient current spikes required to overcome the 14 *Nm* physical stiction and enforce accurate position tracking. Consequently, the policy transfers successfully to the physical hardware without the computational overhead of simulating complex, temperature-dependent friction models).

- θ_{target} - the desired joint angle commanded by the PPO policy.
- $\theta_{current}$ - the actual physical angle of the joint at the current timestep.
- $\dot{\theta}_{current}$ - the current angular velocity of the joint.

The mathematical formulation above ensures that the virtual motors behave like physical hardware. As the joint approaches the target position commanded by the AI, the proportional error (the difference between the neural network's target joint position and the actual joint position) shrinks, and the damping term dominates to smoothly brake the limb, preventing unrealistic, instantaneous movements that would shatter physical gears. This sequential flow from the neural network's target position to the final joint torque is illustrated in Figure 8.

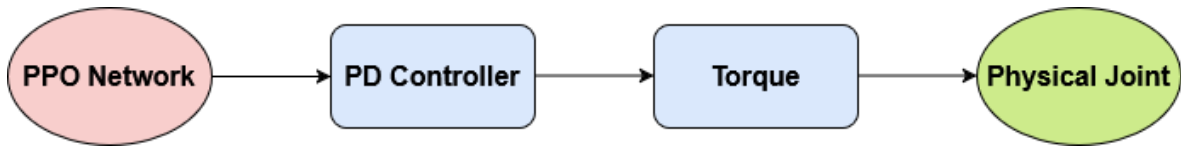


Figure 8: The actuator control pipeline

3.2.2 Kinematic and Dynamic Constraints

To ensure the policies trained within the NVIDIA Isaac Lab environment are viable for real-world deployment, the digital twin must be strictly bounded by the physical limitations of the hardware. The most critical of these boundaries are the actuator limits, specifically the maximum joint torque and radial velocity.

The physical quadruped is driven by Maxon EC 90 flat ($\varnothing 90\text{ mm}$, brushless, 600 W) DC motors paired with Harmonic Drive strain wave gears (CSG-25-160-2UH: 160:1 reduction ratio, 2.5-inch pitch diameter) at each joint. While the physics simulator is capable of generating infinite torque to force the entire 64.36 kg quadruped (comprising a 14 kg central chassis and four articulated legs) forward, the digital twin enforces a hard torque saturation limit of 229.0 Nm to perfectly mirror the physical gearbox's limit for repeated peak torque.

Furthermore, the maximum angular velocity of the joints is mathematically constrained by the motor's absolute mechanical ceiling and the reduction ratio of the gearbox. With the motor possessing an absolute maximum mechanical speed of 5000 RPM , the maximum radial velocity limit enforced in the simulation is derived by applying the gear reduction and converting the output to radians per second:

$$\omega_{max} = \left(\frac{\text{Motor RPM}}{\text{Gear Ratio}} \right) \cdot \left(\frac{2\pi}{60} \right) = \left(\frac{5000}{160} \right) \cdot \left(\frac{2\pi}{60} \right) \approx 3.27\text{ rad/s}$$

By strictly enforcing this velocity limit and the 229.0 Nm torque limit, the PPO algorithm is prevented from synthesizing unphysical, explosive movements. The AI is forced to learn highly efficient, low-torque gaits that the physical motors can actually execute without overheating or stripping the mechanical gears.

3.2.3 Markov Decision Process (MDP) Formulation

Because the robot's locomotion controller is trained using Proximal Policy Optimization (PPO), the interaction between the digital twin and the artificial agent must be formally defined

as a continuous-time Markov Decision Process (MDP). This mathematical framework is defined by the tuple $\langle S, A, P, R, \gamma \rangle$, where the agent observes the state of the simulator at each timestep and outputs an action to maximize its cumulative reward.

The State Space (S_t)

The State Space defines the exact sensory information the neural network "feels" at any given control step t . To bridge the Sim-to-Real gap, the digital twin restricts this observation vector strictly to proprioceptive data that can be reliably captured by the physical hardware's internal IMU (Inertial Measurement Unit) and joint encoders. The continuous observation vector (totaling 235 dimensions) includes:

- Base linear and angular velocities (6 dimensions).
- Projected gravity vector (chassis orientation) (3 dimensions).
- Velocity commands dictating target speed and yaw (3 dimensions).
- The 12 current joint positions ($\theta_{current}$) and velocities ($\dot{\theta}_{current}$) (24 dimensions).
- The previous actions taken at $t - 1$ (12 dimensions). This is a critical Sim-to-Real component that allows the network to implicitly infer mechanical latency and dynamically adapt to the actuator degradation injected during Domain Randomization.
- Local terrain height scan (187 dimensions). A ray-cast grid sampled from the `base_link` to map the topographical geometry beneath the robot's feet. While topologically redundant on a perfectly planar surface, this inherited sensor array is actively maintained to ensure the neural network's input architecture remains

strictly identical to the rough-terrain baseline, allowing for seamless future policy transfer.

The Action Space (A_t)

The Action Space defines the physical commands the neural network is permitted to send to the robot. In alignment with modern quadrupedal control standards, the policy does not output raw joint torques, as doing so frequently results in high-frequency chattering and mathematically unstable gaits.

Instead, the Action Space $A_t \in \mathbb{R}^{12}$ is defined as a 12-dimensional continuous vector representing the target joint positions (θ_{target}). At each timestep, the neural network outputs these 12 target angles, which are subsequently routed to the low-level Proportional-Derivative (PD) controllers described in section 3.2.1. This hierarchical architecture—where the neural network handles high-level trajectory planning (positions) and the PD controller handles low-level physics execution (torques)—ensures that the resulting locomotion is both dynamically stable and mechanically safe for the physical actuators.

3.3 Implementation

The mathematical architecture and digital twin were realized using a multi-layered software stack built primarily in Python. At the foundational physics level, this stack utilizes the NVIDIA Isaac Lab framework to simulate gravity and rigid-body collisions. Moving up the software hierarchy, it leverages the `isaacsim_tasks` library to construct the continuous control environments. These environments act as the simulated physical arenas where the training occurs. Crucially, the 'continuous control' designation refers strictly to the mathematical action space, not the time domain. It denotes that at each discrete time step, the neural network governs the system by outputting precise, floating-point values—specifically, a 12-dimensional array of target joint positions—rather than selecting from a limited set of discrete actions.

These environments are ultimately managed via Gymnasium—an industry-standard Application Programming Interface (API) that acts as the communication bridge between the layers. Specifically, this API translates the complex 3D physics calculated by the simulator into standardized numerical observation vectors (a 235-dimensional array containing data such as the robot's base linear velocity, projected gravity, joint states, and a local terrain height scan) for the neural network, while simultaneously converting the network's continuous action outputs back into physical motor commands.

While the system was architected to support multiple terrain types, the final locomotion policy evaluated in this study was synthesized entirely within the bounded flat-terrain curriculum (`Merkovale-Flat-v0`) to isolate and master baseline biomechanics—namely, maintaining a stable

walking gait while tracking a commanded forward velocity—prior to hardware transfer. The continuous control loop and data flow across this software stack are illustrated in Figure 9.

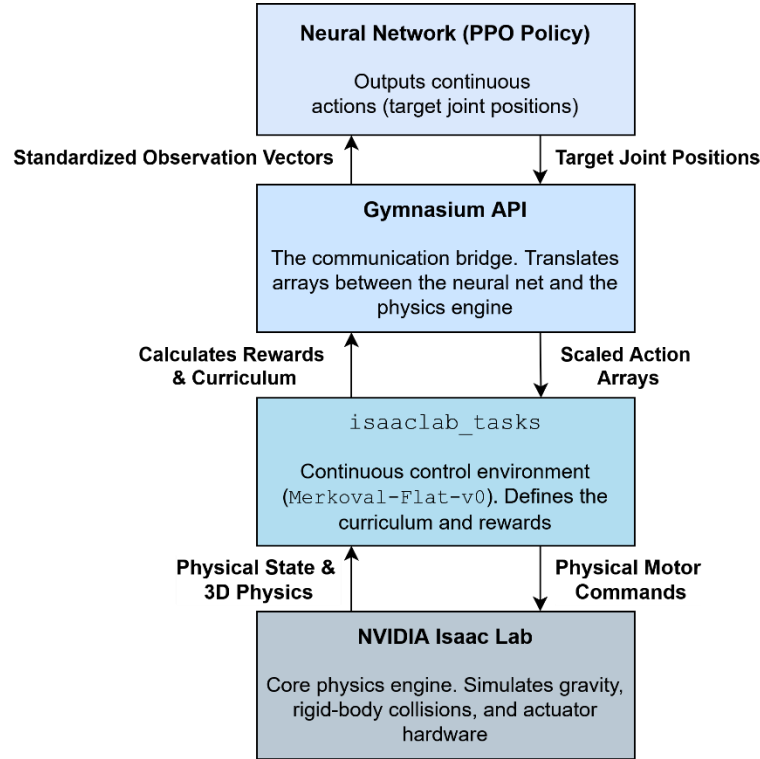


Figure 9: The software architecture of the digital twin, illustrating the continuous data flow between the PPO neural network and the Isaac Lab physics engine, mediated by the Gymnasium API and custom task environments.

Asset Initialization and Actuator Configuration

The physical robot asset was imported into the physics engine as a Universal Scene Description (USD) file (`merkovall.usd`). To initialize training episodes dynamically, the digital twin is spawned with a nominal vertical offset of 0.8 meters.

Within the environment's `DCMotorCfg` module, the actuator parameters are strictly bifurcated into absolute hardware constraints and nominal baseline values. To mathematically prevent the policy from exceeding hardware tolerances, the effort and velocity saturation limits are hardcoded as immutable boundaries (229.0 Nm and 3.27 rad/s , respectively). Conversely, the active Proportional-Derivative (PD) tracking gains—specifically a proportional stiffness (K_p) of 800.0 and a derivative damping (K_d) of 50.0 —alongside the passive internal joint friction (3.5 Nm), are instantiated strictly as nominal starting values. All variables designated as "nominal" in this initialization phase represent the theoretical ideal of the hardware; establishing them is a critical prerequisite, as they serve as the foundational mathematical centers for the subsequent Domain Randomization (DR) algorithms.

Curriculum Architecture and Reward Shaping

From a software architecture perspective, the environment utilizes object-oriented class inheritance. The foundational reward economy, physical boundaries, and domain randomization pipelines were programmed into a base class (`MerkovalRoughEnvCfg`). The final flat-terrain training environment (`MerkovalFlatEnvCfg`) directly inherits this framework, retaining its rigorous hardware safety constraints while overriding specific topographical and kinematic parameters to isolate the fundamental biomechanics of a forward trot.

1. Foundational Hardware Regularization (Inherited Economy)

Operating within this inherited framework, the base locomotion policy is driven by a shaped reward function designed to track target commands while aggressively penalizing erratic, hardware-damaging behaviors. As a foundational training constraint, the velocity command

generator is strictly bounded: lateral (Y –axis) and angular (Z –axis) velocities are locked to 0.0, while forward linear velocity is constrained between 0.4 and 0.6 m/s . This restriction acts as a deliberate dimensionality reduction strategy for the reinforcement learning process; by eliminating omnidirectional variables, it prevents the untrained policy from converging on sub-optimal local minima (e.g., spinning in place or lateral avoidance) and forces the neural network to isolate and solve the pure biomechanical challenges of sagittal-plane locomotion, such as vertical foot clearance and impact absorption over rough terrain. The policy heavily prioritizes tracking this forward speed (`track_lin_vel_xy_exp`, weight: 15.0) alongside basic gait articulation. Crucially, to prevent the policy from converging on an inefficient "sliding" locomotion, the economy explicitly rewards the robot for executing a proper stepping motion (`feet_air_time`, weight: 0.01, threshold: 0.25 s). By enforcing a strict 0.25-second airborne threshold, this function prevents the neural network from exploiting the reward through rapid, high-frequency foot tapping; it mathematically mandates a deliberate, sustained swing phase.

However, optimizing purely for kinematic performance in simulation often leads to hardware-destroying behaviour on the physical robot. To bridge this sim-to-real gap and ensure safe deployment, the environment simultaneously enforces the following foundational hardware regularization penalties:

- **Actuator Abuse Prevention:** To prevent the neural network from exploiting the physics engine or commanding explosive forces that would strip the physical

Harmonic Drives, a strict torque regularization penalty (`dof_torques_l2`, weight: -0.000010) is continuously applied.

- **Signal Chattering Reduction:** High-frequency, erratic motor commands cause severe overheating in physical DC motors. To enforce smooth continuous motion, penalties are applied to rapid changes in the action output (`action_rate_l2`, weight: -0.15) and joint acceleration (`dof_acc_l2`, weight: $-2.5e - 7$).
- **Undesired Contact Penalty:** To prevent the robot from dragging its body or collapsing, a heavy penalty (`undesired_contacts`, weight: -10.0) is instantly triggered if any part of the robot other than the designated foot geometries (i.e., the base link, hips, thighs, or calves) makes contact with the ground.
- **Z-Axis Bouncing:** A linear vertical velocity penalty (`lin_vel_z_l2`, weight: -2.0) discourages the policy from developing highly inefficient, "bouncing" gaits that waste kinetic energy vertically rather than translating it forward.

2. Flat-Terrain Gait Specialization (Environment Overrides)

Building upon the safe hardware baseline and inherited velocity constraints (lateral and angular locked to 0.0, forward bounded between 0.4 and 0.6 m/s), the `MerkovaFlatEnvCfg` environment applies specific overrides to drive the synthesis of a stable quadrupedal walking gait. First, the procedural terrain curriculum inherited from the base environment was explicitly disabled (`self.curriculum.terrain_levels = None`). The simulation scene was then reconfigured to utilize a continuous planar surface by setting the `terrain_type` attribute to

"plane" and systematically nullifying the `terrain_generator`. Furthermore, the ground plane was assigned a custom physical material utilizing the `isaacsim` library's `RigidBodyMaterialCfg` class, which explicitly locked restitution (bounciness) to 0.0. This completely removes variable bounce mechanics and vertical chatter from the environment. However, to ensure the resulting gait remains robust against unpredictable surfaces, the inherited Domain Randomization was left active, systematically scrambling the static and dynamic friction coefficients between 0.3 and 1.25 at the start of every episode. This forces the policy to master slip-recovery on a planar surface without the confounding noise of restitution.

To force the AI to focus on deliberate stepping mechanics rather than raw speed, the primary velocity tracking reward was heavily scaled down (`track_lin_vel_xy_exp`, weight: 3.0), while the swing-phase reward was significantly increased (`feet_air_time`, weight: 1.0) to enforce a pronounced, proper stepping motion. In complex or uneven environments, quadrupedal policies require significant angular freedom to negotiate obstacles and varying inclines. However, on perfectly planar terrain, extreme angular deviations are unnecessary and typically indicate convergence on an inefficient local minimum. To regulate the overall angular attitude of the chassis, a targeted orientation penalty (`flat_orientation_l2`, weight: -7.0) is applied. Mathematically, this constraint functions as a rigid stricture, penalizing any deviation from a perfectly flat, horizontal plane to suppress destabilizing angular deviations.

Furthermore, to regulate a postural anomaly observed during early uncalibrated runs, a custom programmatic penalty was introduced. Initial policies frequently converged on an asymmetric, sidewinding gait, where the quadruped's hind legs maintained a persistent lateral

skew. To completely suppress this behaviour, a targeted Hip-Roll Penalty (Coronal Plane Constraint) was implemented. This joint posture regularization isolates the hip-roll actuators and applies a squared error penalty (weight: -2.0) for any deviation from a 0.0 -radian baseline. Mathematically, this constraint functions as a rigid structure explicitly designed to eliminate lateral drift and force the legs into a strictly parallel alignment. To synthesize all of these mathematical penalties and curriculum constraints into a functional software stack, the environment relies on the previously mentioned class inheritance structure. The specific terrain overrides—ranging from the nullified procedural generation to the targeted hip-roll penalty—are cleanly encapsulated within the `MerkovalFlatEnvCfg` class. The complete object-oriented hierarchy, detailing how these specialized constraints are inherited and applied on top of the base physics framework, is visualized in Figure 10.

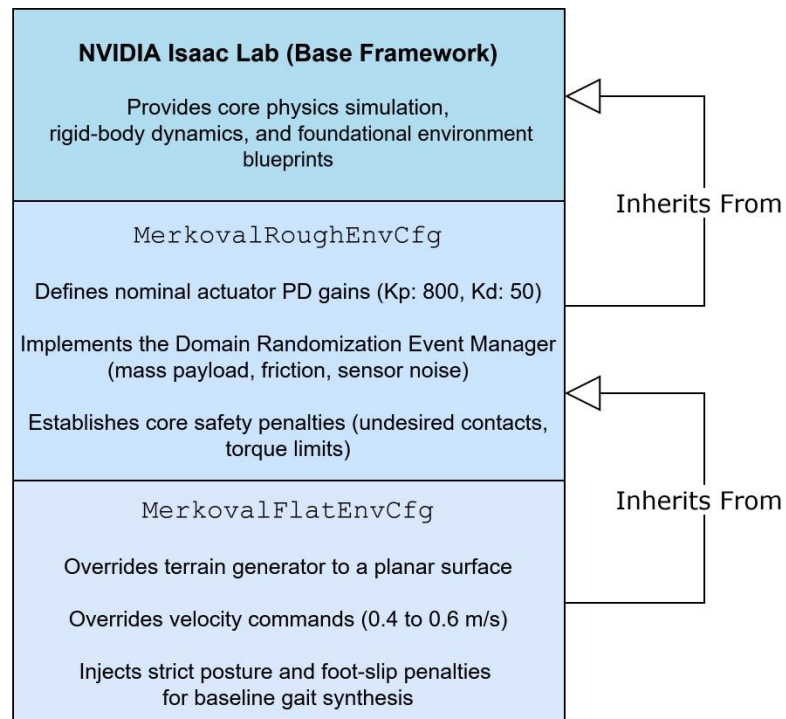


Figure 10: UML class inheritance diagram demonstrating the object-oriented architecture used to configure the digital twin environments.

Domain Randomization Pipeline

Because the flat environment inherits the base configuration, it retains a comprehensive Domain Randomization (DR) pipeline governed by a dedicated Event Manager. This programmatic architecture bridges the Sim-to-Real gap by ensuring the neural network never controls the exact same mechanical system twice.

The majority of the domain shifts are explicitly mapped to the environment's `reset` mode. Every time an episode terminates and the digital twin respawns, the `EventTerm` module pulls new physical parameters from a uniform distribution. This includes randomizing the initial vertical drop-height between 0.76 and 0.85 meters, forcing the policy to dynamically catch the robot's weight from unpredictable elevations. Furthermore, it subjects the main chassis (`base_link`) to dynamic payload masses (-5.0 kg to $+15.0\text{ kg}$) accompanied by randomized Center of Mass shifts ($\pm 0.05\text{ m}$ across all axes), while, as mentioned previously, heavily randomizing the physical floor friction between 0.3 and 1.25.

To simulate internal mechanical wear, custom event terms (`mdp.randomize_actuator_gains` and `mdp.randomize_joint_parameters`) are injected to episodically scale the actuator's baseline parameters. Specifically, the proportional stiffness (800.0) and derivative damping (50.0) are scaled by $\pm 15\%$, while the internal dry friction (3.5 Nm) is scaled by $\pm 25\%$.

Beyond these initial respawn parameters, the pipeline also simulates active environmental hazards. Physical perturbations are injected using a timed interval trigger (`mdp.push_robot`) that

applies sudden two-dimensional velocity impulses ($\pm 1.5 \text{ m/s}$) to the chassis while walking to test dynamic recovery. Finally, crucial sensor degradation is handled by an explicit `enable_corruption = True` boolean, which continuously applies Gaussian noise over the simulated IMU and joint encoders to prevent the AI from overfitting to perfect simulator data.

Hardware and Training Architecture

The control policy was trained using the `RSL_RL` implementation of Proximal Policy Optimization (PPO). The actor-critic network architecture (`MerKovalFlatPPORunnerCfg`) was designed with identical hidden layer dimensions of `[512, 256, 128]`, built and trained using PyTorch (an open-source machine learning framework used to construct the neural network). The algorithm was configured to run 24 steps per environment per update cycle. All neural network training and physical simulations were accelerated using an NVIDIA RTX 2080 Ti GPU, allowing the system to simulate thousands of parallel environments simultaneously to achieve high sample efficiency during gait synthesis.

4. EXPERIMENTAL RESULTS

4.1 Methodology

To evaluate the efficacy of the locomotion model developed in Chapter 3, a rigorous training and evaluation framework was established. The experimental setup was designed first to synthesize a stable walking policy, and subsequently to measure its robustness, efficiency, and tracking accuracy.

Training Protocol

The agent was trained using the Proximal Policy Optimization (PPO) algorithm within the simulated `Merkoval-Flat-v0` curriculum. To ensure full policy convergence, the training cycle was executed for a total of 1500 iterations. Throughout this process, the policy was continuously subjected to the active Domain Randomization pipeline detailed in Section 3.3 to simulate real-world physical stress.

Evaluation Metrics

To objectively assess the viability of the resulting gait for physical deployment, the policy was evaluated against specific quantitative metrics logged during the training cycle. These metrics are categorized into four core areas of performance:

1. **Algorithm Convergence and Survivability:** Tracking the overall mean reward (`mean_reward`) and average episode length (`mean_episode_length`) over the 1500 iterations to verify stable learning without experiencing policy collapse. Furthermore,

because the simulation is explicitly programmed to terminate an episode prematurely if the robot's central chassis comes into physical contact with the ground, the mean episode length serves as a direct measurement of the agent's ability to survive the full episode duration despite continuous physical perturbations and the chaotic exploration inherent to the early stages of reinforcement learning.

2. **Kinematic Performance and Tracking:** Evaluating the primary velocity tracking accuracy (`lin_vel_xy`) to verify the robot effectively matches the commanded forward speed. This is analyzed alongside the gait articulation reward (`feet_air_time`) to quantify the policy's kinematic strategy—specifically, observing how the agent balances the theoretical ideal of a sustained swing phase against the dynamic stance-phase adaptations required to sustain locomotion under maximum physical load.
3. **Hardware Regularization and Safety:** Quantifying the physical demands placed on the simulated hardware by analyzing torque usage (`dof_torques_l2`) and high-frequency motor chattering (`action_rate_l2`) ensures strict policy adherence to the physical kinematic boundaries of the chassis. Finally, penalty logs for body clearance (`undesired_contacts`) are evaluated to guarantee spatial clearance, thus avoiding catastrophic ground collisions.
4. **Postural Regulation:** Assessing the policy's adherence to required kinematic alignments by analyzing penalty logs for lateral hip-roll posture (`hip_posture`) and

overall Chassis Orientation (`flat_orientation_l2`, which mathematically penalizes deviations from a stable pitch and roll).

4.2 Results

The empirical data analyzed throughout this section was continuously logged during the 1500-iteration training cycle using TensorBoard, an industry-standard telemetry toolkit. The PPO algorithm's real-time reward metrics and episodic variance were subsequently extracted and plotted, overlaid with smoothed trendlines to clearly demonstrate the overall trajectory of policy convergence.

4.2.1 Algorithm Convergence and Survivability

The foundational metric for any Reinforcement Learning control policy is its ability to reliably converge on a stable solution without succumbing to catastrophic collapse.

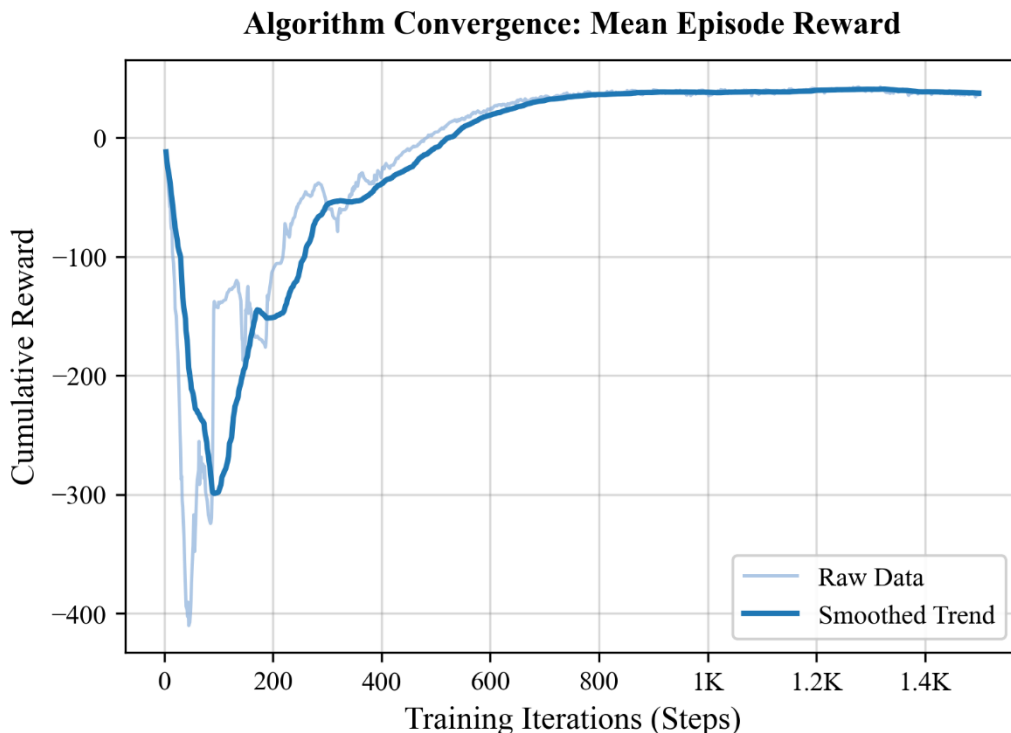


Figure 11: Algorithm convergence demonstrating the mean episodic reward over the 1500-iteration training cycle.

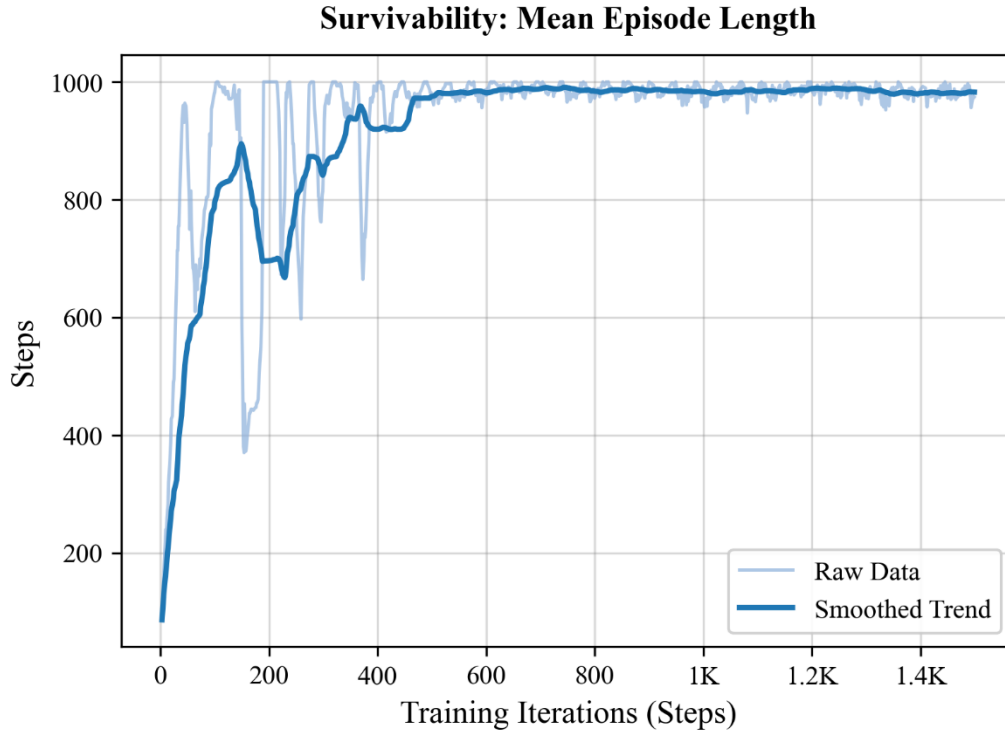


Figure 12: Mean episode length over the 1500-iteration training cycle, demonstrating the agent's overall survivability.

As illustrated in the convergence data, the policy exhibited standard exploratory instability during the first 200 iterations, characterized by severe negative cumulative rewards and short episode lengths. This indicates the untrained agent was frequently triggering terminal states (falling over) while attempting to navigate the simulated physics. However, the PPO algorithm demonstrated robust learning, with the mean reward climbing sharply and plateauing near iteration 800. Crucially, the mean episode length locked onto the maximum 1000-step limit. This definitively proves the agent successfully synthesized a stable control policy, maintaining dynamic balance and surviving the full duration of the simulation without triggering catastrophic failure.

4.2.2 Kinematic Performance and Tracking

Survivability must be paired with accurate task execution. The primary objective of the policy was to synthesize a forward-translating gait that accurately tracked commanded velocities.

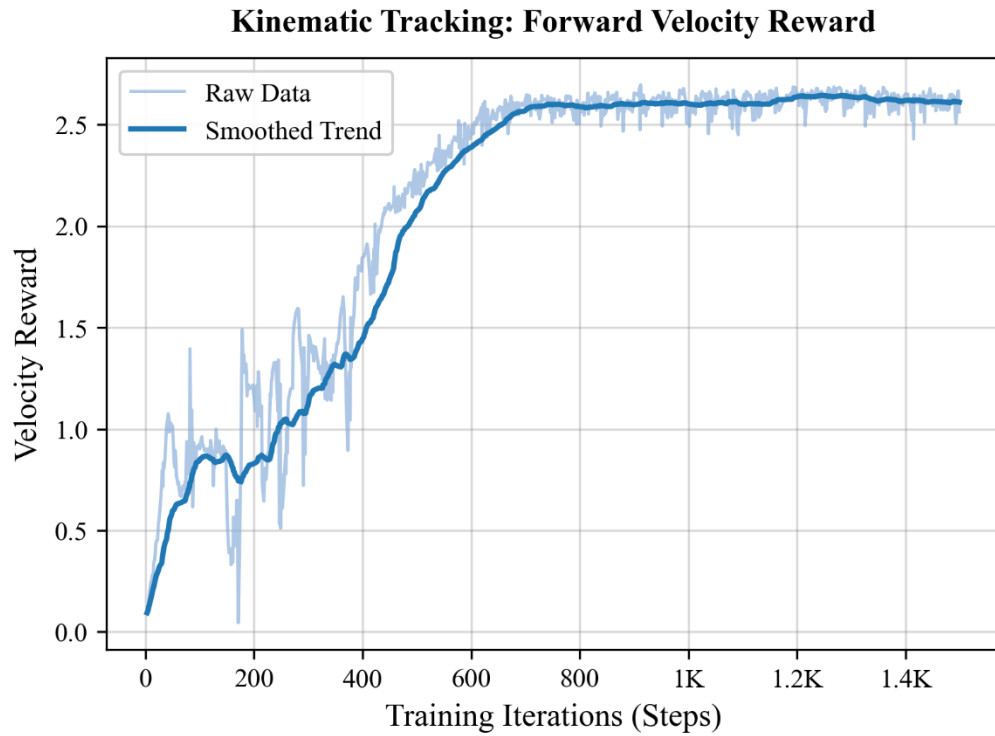


Figure 13: Kinematic tracking demonstrating the forward velocity reward over the 1500-iteration training cycle.

As illustrated in the forward velocity reward telemetry, the policy demonstrates excellent command adherence. Following the initial exploratory phase characterized by standard variance, the reward gradient climbs rapidly and stabilizes near the maximum theoretical value. This confirms that the agent successfully learned to propel the high-inertia chassis forward and tightly track the maximum 0.6 m/s target velocity. However, achieving this sustained momentum under

maximum physical payload required a highly specific mechanical adaptation, which is revealed by analyzing the secondary gait articulation metric.

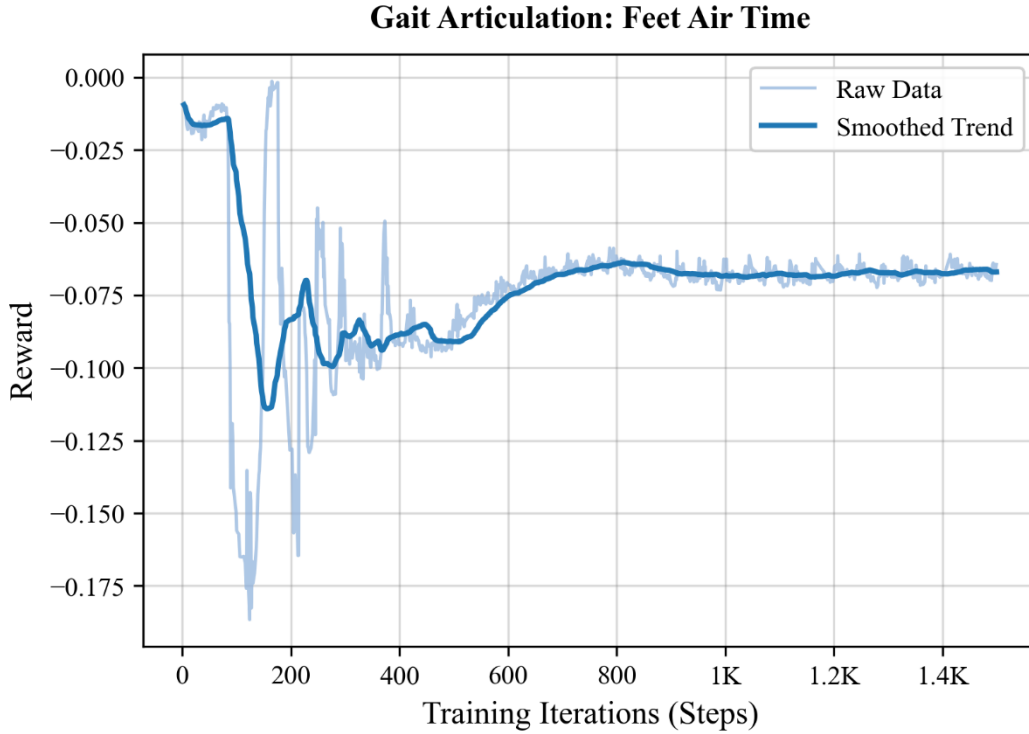


Figure 14: Gait articulation demonstrating the feet air-time reward over the 1500-iteration training cycle.

As shown in Figure 13, the `feet_air_time` reward experiences significant initial volatility before converging and stabilizing at a constant negative baseline of approximately -0.075 . As established in the environment overrides (Section 3.3), this metric is programmed with a strict 0.25-second threshold and a weight of 1.0. Because the reward function computes the sum of $(\text{last_air_time} - \text{threshold})$ across all feet contacting the ground at each timestep, this stabilised value represents a consistent aggregate timing deficit across the active foot contacts. Regardless of the precise gait pattern, this permanent negative baseline confirms that the policy consistently failed to achieve the required 0.25-second aerial duration despite explicit reward incentive.

4.2.3 Hardware Safety and Regularization

A critical barrier in Sim-to-Real deployment is the "reality gap," where policies that succeed in simulation destroy physical hardware due to infinite torque allowances. To evaluate deployment readiness, strict hardware regularizations were analyzed.

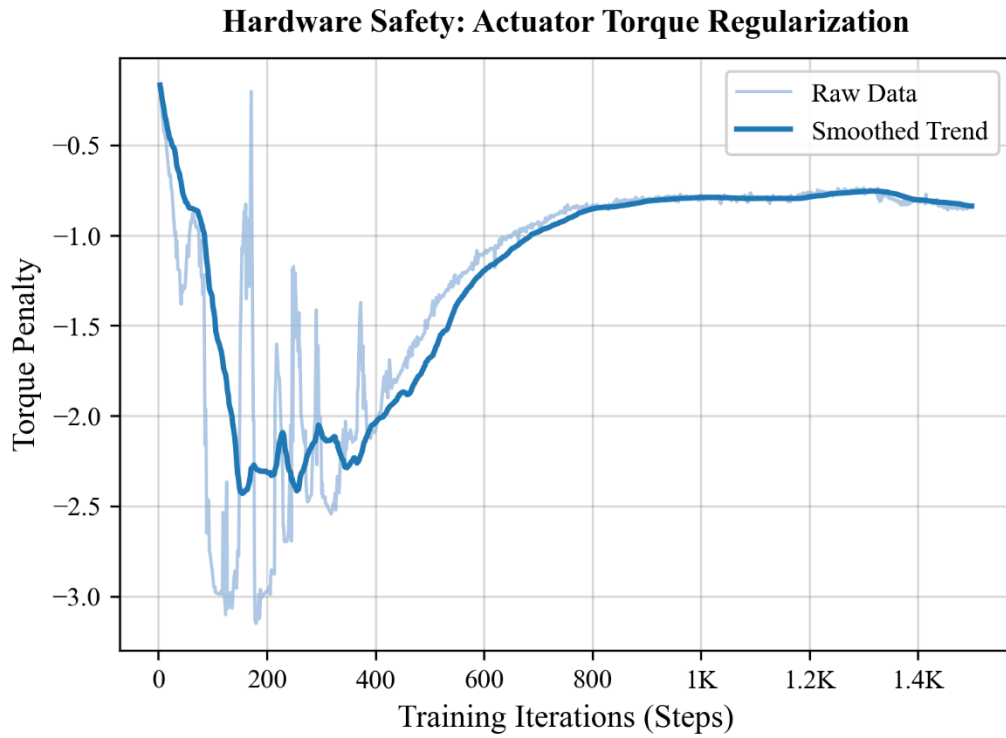


Figure 15: Hardware safety demonstrating the actuator torque regularization penalty over the 1500-iteration training cycle.

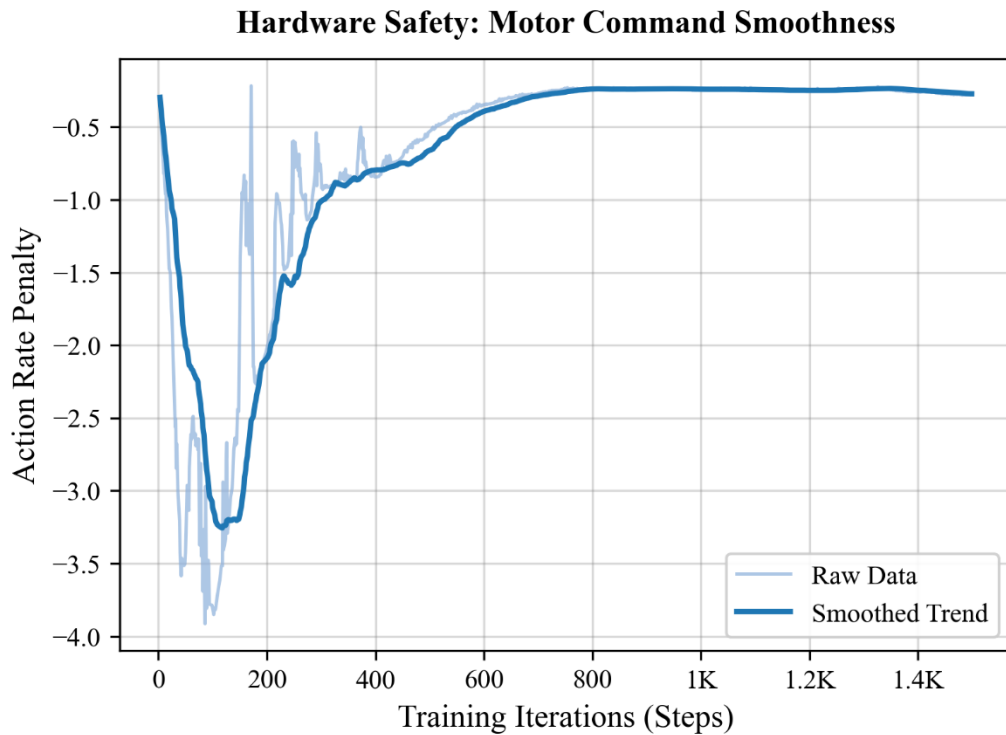


Figure 16: Hardware safety demonstrating the motor command smoothness (action rate penalty) over the 1500-iteration training cycle.

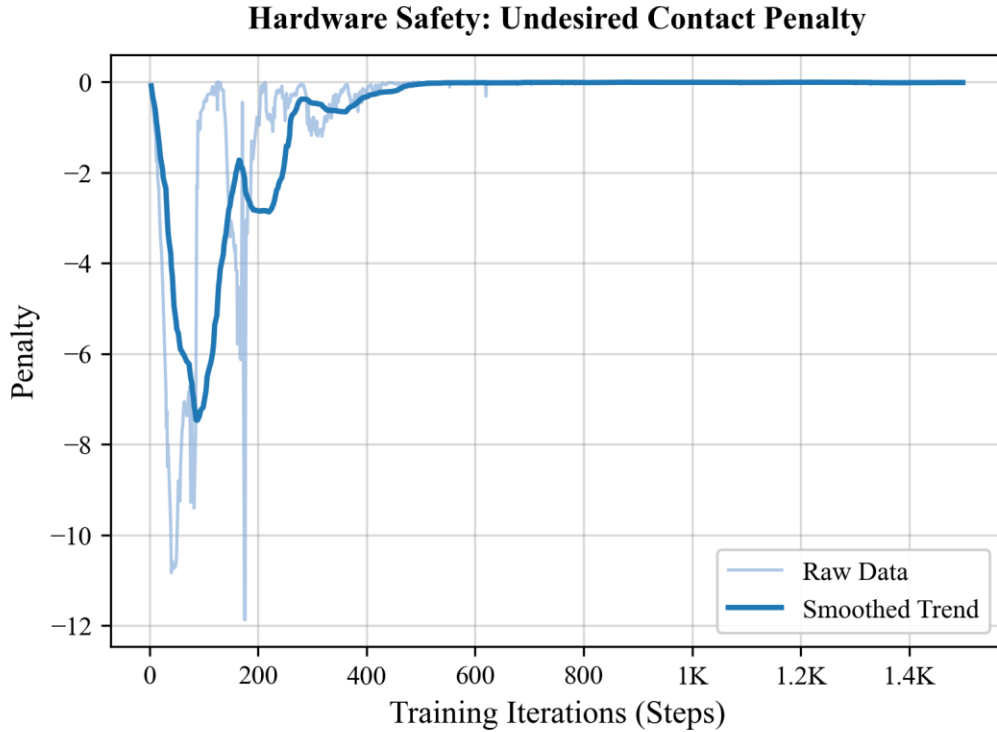


Figure 17: Hardware safety demonstrating the undesired contacts penalty over the 1500-iteration training cycle.

The data reveals a stark contrast between early and late-stage training behaviour. Initially, the untrained agent triggered severe negative penalties across all three hardware safety metrics. The Action Rate and Torque logs display deep initial deficits, indicating high-frequency motor oscillations and peak joint forces. Concurrently, the Undesired Contact penalty registered severe negative spikes, indicating frequent ground collisions involving non-foot bodies.

As training progressed, the programmatic regularizations forced the policy to adopt a controlled kinematic profile. The torque and action rate penalties smoothly converged and stabilized at marginal residual values. Furthermore, the `undesired_contacts` penalty flattened

entirely to zero by iteration 400 and maintained this zero-baseline for the remainder of the training cycle, confirming that no further prohibited ground contacts were logged by the simulation engine.

4.2.4 Postural Regulation

To enforce general torso stability and address specific kinematic anomalies—namely, the tendency for the quadruped to adopt an asymmetric, sidewinding gait—a combination of standard and custom postural constraints were evaluated.

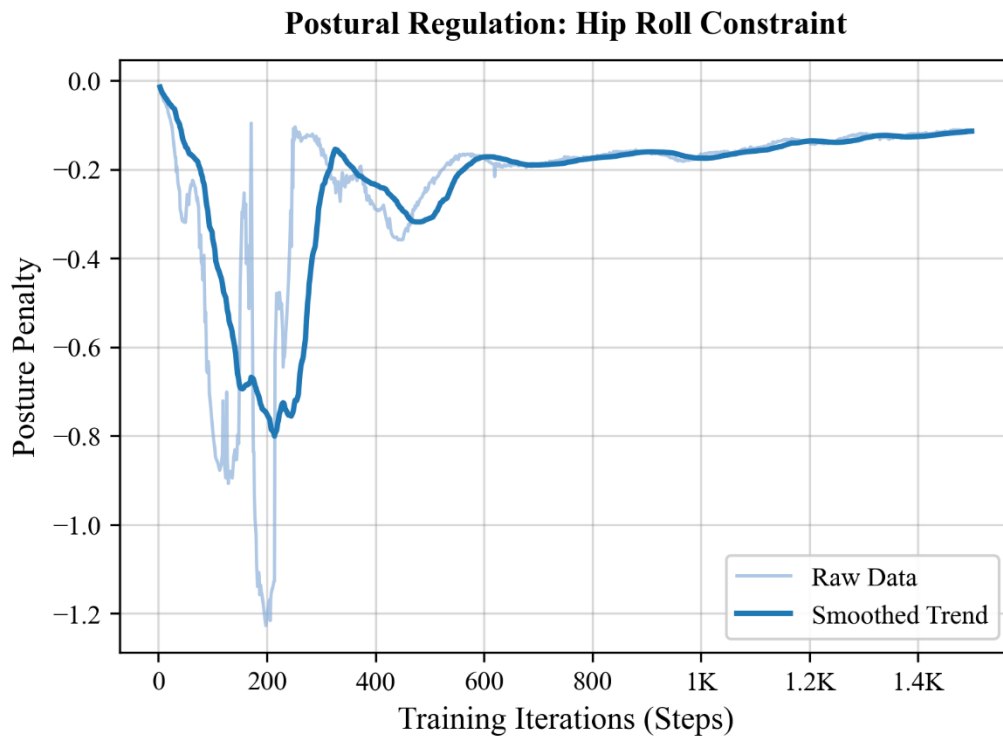


Figure 18: Postural regulation demonstrating the hip-roll constraint penalty over the 1500-iteration training cycle.

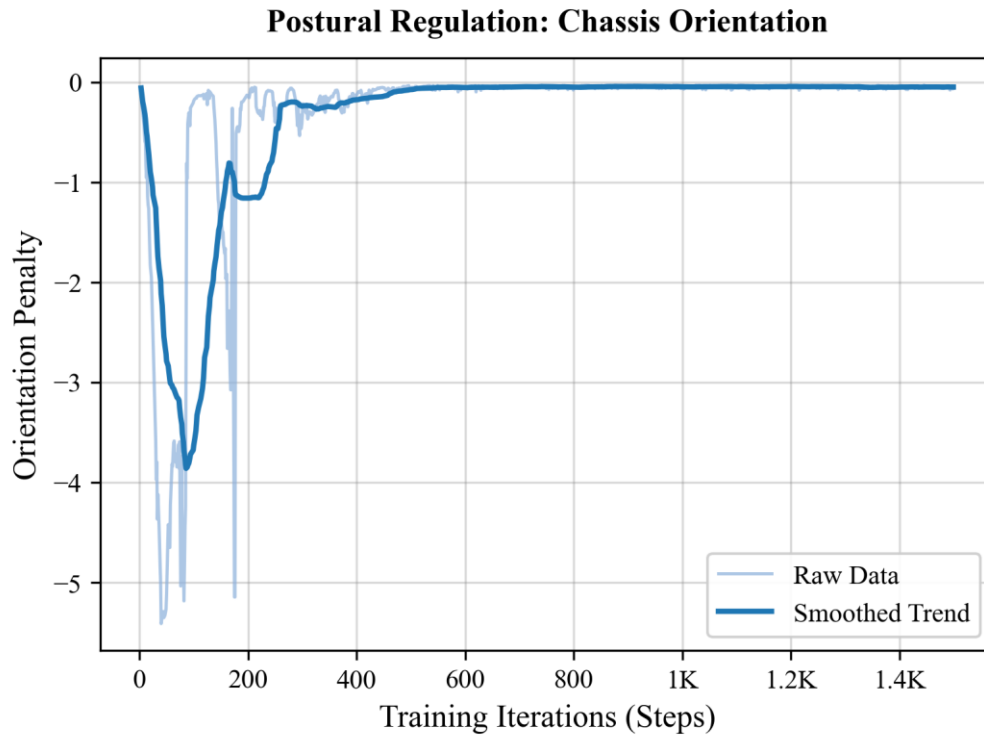


Figure 19: Postural regulation demonstrating the chassis orientation penalty over the 1500-iteration training cycle.

To evaluate the structural alignment of the robot during locomotion, the Coronal Plane Constraint (`hip_posture`) and Chassis Orientation penalties were tracked during the training cycle. As shown in the respective telemetry logs, both metrics exhibited significant negative penalties during the first 300 iterations, indicating severe lateral skew and body deviation. Following this initial exploratory phase, both the hip-roll penalty and the orientation penalty rapidly converged. By iteration 600, both metrics stabilized at a marginal residual value near zero, maintaining this baseline for the remainder of the 1500-step training cycle.

4.2.5 Dynamic Disturbance Rejection and Robustness

While the training convergence metrics detailed in previous sections prove the mathematical stability of the policy, deployment in hazardous environments—such as nuclear decommissioning sites—demands empirical proof of physical resilience. To evaluate the final policy’s robustness, an isolated time-series disturbance test was conducted. The trained agent was commanded to maintain its target forward crawl, and a sudden 1.5 m/s lateral velocity perturbation (simulating falling debris or a severe physical collision) was injected directly into the chassis mid-gait.

To capture this high-fidelity data, a custom telemetry pipeline was developed to override the standard evaluation environment. Rather than relying on qualitative visual observation, a dedicated script (`play_telemetry.py`) was engineered to log real-time kinematic states—including target angles, actuator torques, and forward velocity—at every simulation timestep. Following the simulation, an automated detection algorithm (`plot_robustness.py`) mathematically isolated the exact moment of peak deceleration (the impact event) and extracted a strict 10-second analytical window. This completely eliminates human bias in the data cropping process and provides an objective, reproducible visualization of the agent's dynamic recovery.

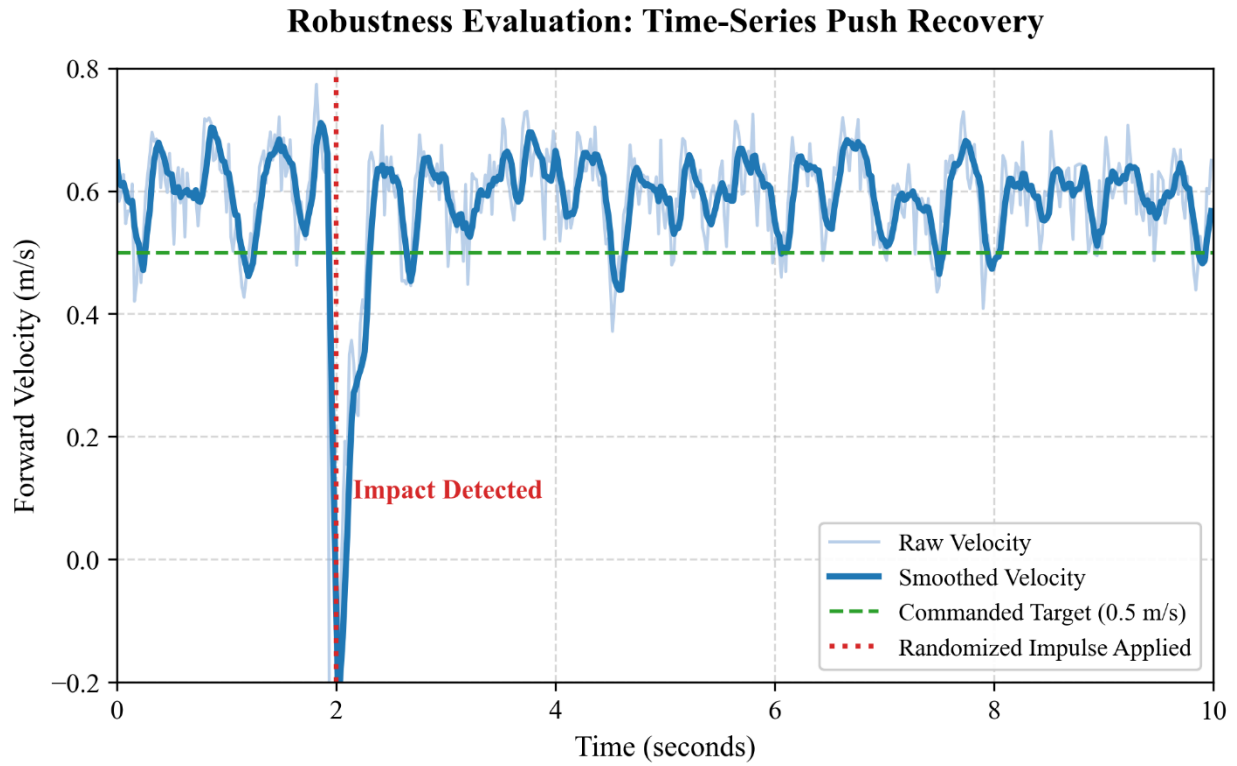


Figure 20: Robustness evaluation demonstrating the time-series push recovery telemetry.

As shown in the time-series telemetry, the quadruped maintained a continuous forward velocity prior to the perturbation. At exactly 2.0 seconds, the injected perturbation triggered an immediate and severe drop in forward velocity, briefly driving the metric into the negative domain. In successful trials, the agent demonstrated dynamic recovery, re-establishing the commanded target speed within approximately 0.5 seconds of the impact. However, supplementary visual telemetry confirmed that this recovery was not absolute, as the identical perturbation still occasionally induced a terminal capsizing state.

To provide a comprehensive visual evaluation of the digital twin's capabilities and physical resilience, three distinct recordings have been compiled. The baseline kinematic tracking data is corroborated in Figure 21, which demonstrates the robot executing the nominal 0.6 m/s target velocity with stable forward translation. Figure 22 demonstrates the agent's successful dynamic recovery and target speed re-establishment following a 1.5 m/s lateral impulse. Conversely, Figure 23 documents the kinematic edge-cases where the identical impulse induces a terminal capsizing failure.

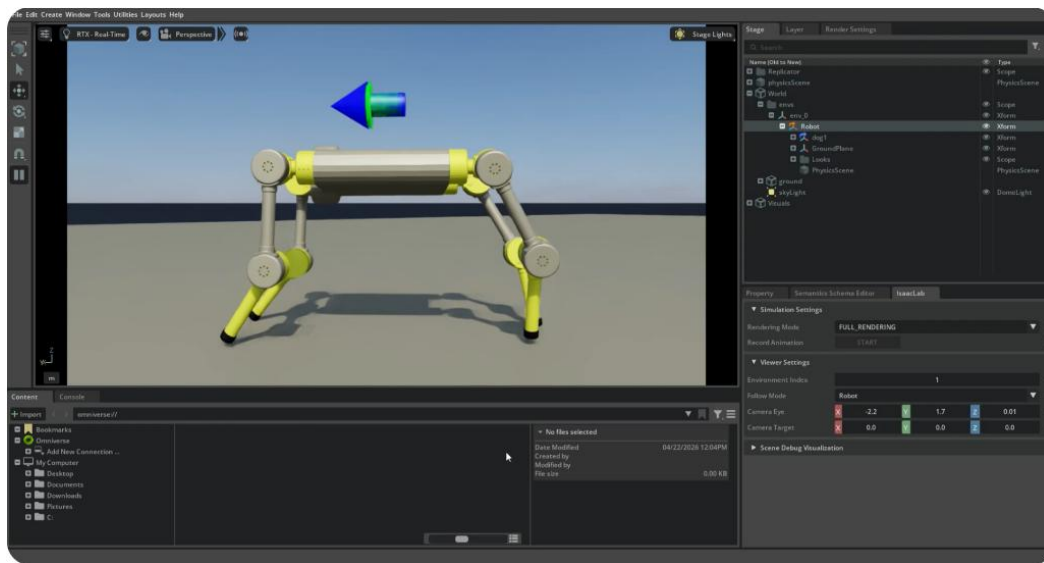


Figure 21: Visual telemetry of the converged policy executing the nominal 0.6 m/s target velocity.

Click the image to view the video.

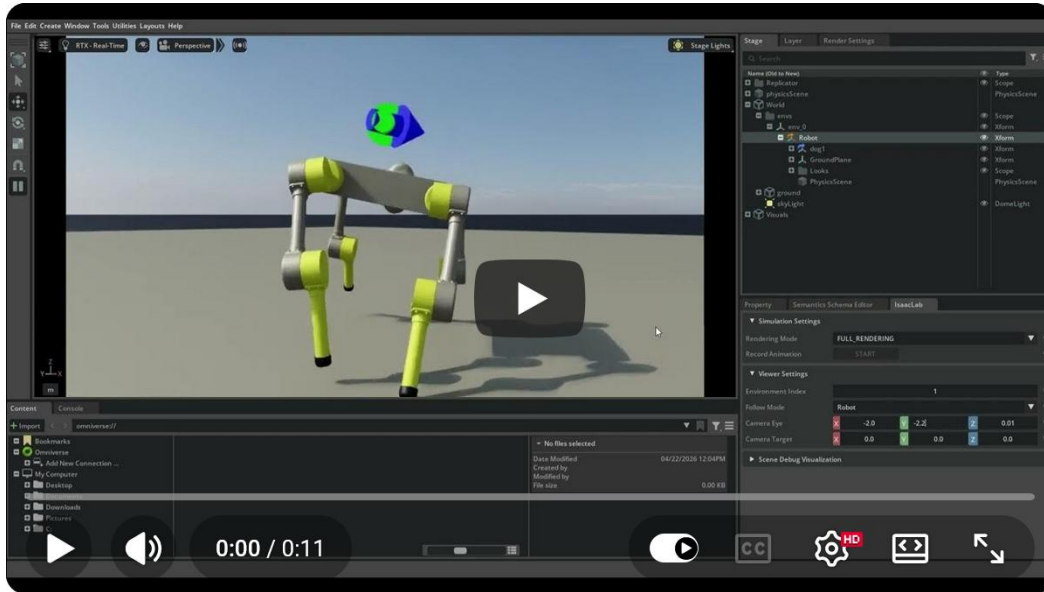


Figure 22: Visual telemetry demonstrating successful dynamic recovery and target speed re-establishment following a 1.5 m/s lateral impulse. **Click the image to view the video.**

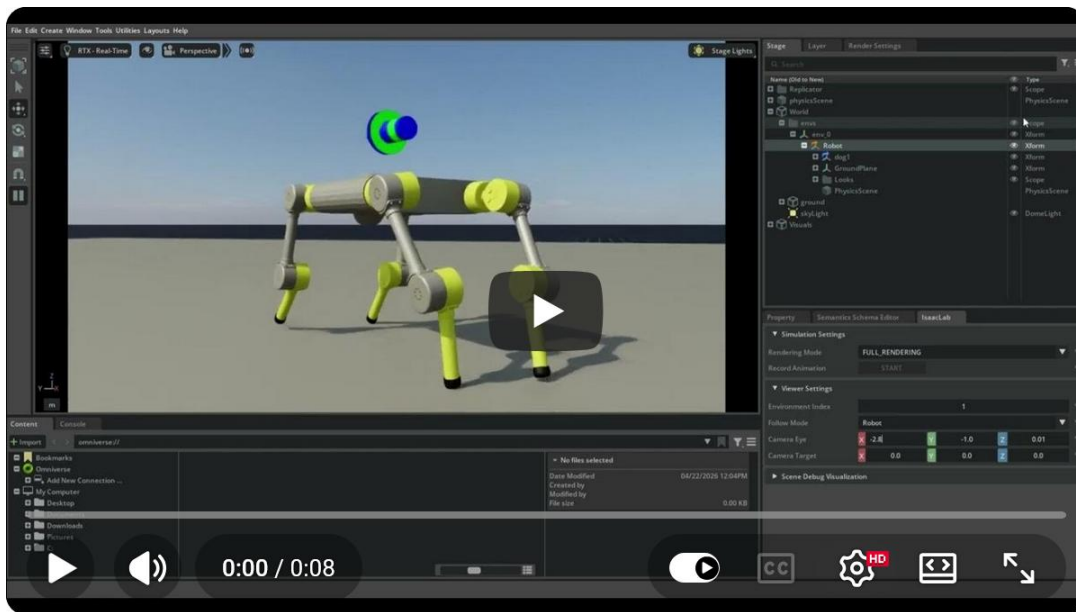


Figure 23: Visual telemetry documenting the kinematic edge-cases where a 1.5 m/s lateral impulse induces a terminal capsizing failure. **Click the image to view the video.**

4.3 Analysis

4.3.1 The Impact of Hardware Regularization on Convergence

The convergence data presented in Section 4.2.1, coupled with the hardware safety metrics in Section 4.2.3, demonstrates the critical efficacy of programmatic regularization. In unconstrained simulation, a reinforcement learning agent will typically exploit infinite virtual torque to maximize its velocity reward, resulting in erratic, "bang-bang" motor commands that would instantly overheat physical actuators and destroy the Harmonic Drive transmissions. By enforcing a strict torque usage penalty alongside an action-rate penalty to punish erratic motor oscillations, the digital twin successfully bottlenecked the AI's options. To survive the training environment, the policy was mathematically forced to abandon destructive exploration and converge on a highly smoothed, energy-efficient kinematic trajectory. The stabilization of these penalties near zero proves that the resulting standard-cadence walk operates safely within the 229.0 *Nm* continuous limits of the physical hardware.

Furthermore, this kinematic regularization extends beyond the internal actuators to the external structural safety of the chassis. As evidenced by the undesired contact penalty flattening entirely to zero, the policy was effectively prohibited from utilizing suboptimal balance exploits—such as dragging its knees or resting its base link on the ground to artificially increase its support polygon. By mathematically eliminating these collision states, the regularization forced the agent to continuously suspend its full 64.36 *kg* system mass strictly through active foot contact. This proves the resulting gait maintains absolute ground clearance, a mandatory prerequisite for avoiding high-centering and mechanical damage during physical deployment.

4.3.2 Kinematic Boundaries and Actuator Optimization

To ensure optimal sim-to-real transfer, the `feet_air_time` reward threshold was not treated as an arbitrary software parameter, but was instead mathematically derived directly from the physical specifications of the hardware. To establish this optimal baseline under maximum kinematic stress, the system's kinematics are analyzed at the upper boundary of the domain randomization target ($v = 0.6 \text{ m/s}$). Assuming a standard trotting duty factor of 0.5—meaning a leg spends exactly half of its total cycle time pushing the ground and half swinging through the air—a 0.25-second swing mandates a total stride cycle of $T_{cycle} = 0.5$ seconds (comprising exactly one 0.25-second swing phase in the air followed by one 0.25-second stance phase on the ground). Achieving the 0.6 m/s velocity target within this timeframe requires a rigid linear stride length (L) of 0.3 meters ($L = 0.6 \cdot 0.5$).

Translating the actuator's angular rotation into this linear stride length is governed by the arc length formula, $\theta = \frac{L}{r}$. Based on the physical SolidWorks assembly, the limb comprises a 0.46 m femur and a 0.40 m tibia. Because continuous dynamic locomotion prohibits operating at full singular extension (180°), the limb must maintain a bent-knee posture for shock absorption. Applying the Law of Cosines to an interior knee angle of 120° — chosen to represent a conservative, deep-stance kinematic stress test — yields an effective kinematic radius of $r = \sqrt{0.46^2 + 0.40^2 - 2(0.46)(0.40)\cos(120^\circ)} \approx 0.745$ meters. To achieve the required 0.3-meter stride at this radius, the thigh joint must sweep an angle of $\theta \approx 0.4$ radians ($\theta = \frac{0.3}{0.745}$).

However, forcing the hardware to execute this fixed 0.4-radian sweep within a strictly bounded 0.25-second time limit introduces an extreme kinematic demand. In this study, locomotion is not governed by explicitly programmed kinematic equations, but by the RL policy. As mentioned prior, the neural network outputs target joint positions, which the lower-level PD controller tracks. However, the RL agent must learn to optimize these trajectories against strict simulated penalties for excessive torque (`dof_torques_l2`) and aggressive action rates (`action_rate_l2`).

In order to evaluate the absolute theoretical limits of the hardware that the AI commands, one must analyze the mathematically optimal trajectory the agent could theoretically learn to minimize these penalties. In quadruped robotics, a standard approach to modeling this optimal baseline is the minimum-jerk (or S-curve) profile (e.g., Zhang et al., 2020)—with "jerk" referring to abrupt changes in acceleration that produce impulse loads capable of shearing the tooth faces of transmission gears (Sakakibara et al., 1990).

A quintic (5th-degree) polynomial models this profile; it is a position equation as a function of time whose six coefficients are uniquely and fully determined by imposing six boundary conditions on the trajectory. Specifically, it constrains the motion to traverse the required distance (θ) by fixing the start and end positions, forcing the start and end velocities to zero ($v_0 = v_f = 0$), and forcing the start and end accelerations to zero ($a_0 = a_f = 0$)—with all velocity and acceleration terms zeroed at the precise moments of liftoff and touchdown.

Trapezoidal profiles exhibit instantaneous jumps in acceleration at transition points—discontinuities whose time-derivative is undefined, producing infinite jerk. Sinusoidal profiles avoid this but fail the zero-acceleration boundary conditions at liftoff and touchdown, meaning acceleration is non-zero at the precise moments of ground contact where impact loads are highest. Unlike these simpler models, 5th-degree polynomial profiles maintain continuous, differentiable velocity and acceleration profiles across the entire swing phase while simultaneously zeroing both quantities at the precise moments of liftoff and touchdown—satisfying all six boundary conditions without sacrificing mechanical safety for computational simplicity.

To cover the required distance (θ) in a set amount of time (t_{swing} , the duration of the swing phase) under this optimized theoretical profile, the limb is strictly bound by the zero-velocity and zero-acceleration boundary conditions at liftoff and touchdown. Because the motor is forced to ramp up from zero and ramp down to zero to satisfy these constraints, it spends the initial and final phases of the swing moving significantly slower than the baseline average speed required across the full duration of the swing. As a direct consequence of these boundary constraints, the peak velocity in the middle of the trajectory must be proportionally higher than the average speed required across the full swing duration. Specifically, the motor's peak velocity must reach exactly 1.875 ($\frac{15}{8}$ in fraction form) times its average speed. This relationship is derived by Biagiotti & Melchiorri (2008) and defined by the kinematic formula: $\omega_{peak} = 1.875 \cdot (\frac{\theta}{t_{swing}})$. Furthermore, this required peak must occur exactly mid-swing. Shifting the peak earlier compresses the acceleration phase, creating an abrupt transition to deceleration that induces a jerk spike at the peak. Shifting it later produces the same discontinuity in reverse—a sudden acceleration-to-deceleration transition under time pressure—generating an equivalent impulse load through the

drivetrain. A mathematically optimal 5th-degree trajectory avoids these spikes by perfectly and symmetrically balancing acceleration across the entire available time window, thereby keeping this mandatory mid-swing velocity surge as low as physically possible. By establishing this theoretical floor—the lowest possible peak velocity required for the most efficient motion—one can prove that if the absolute best-case trajectory exceeds the mechanical ceiling of the actuators, any sub-optimal, hesitant, or asymmetrical policy learned by the AI would effectively waste available swing time, demanding even higher peak velocities to compensate and rendering the motion physically unviable.

By substituting the known operational parameters into this formula, the physical constraints of the current system become mathematically evident. Under the baseline 0.25-second threshold, executing the 0.4-radian sweep yields an average speed of 1.6 rad/s ($\omega_{avg} = \frac{0.4}{0.25}$). Applying the S-curve formula, this results in a required peak velocity of exactly 3.0 rad/s ($\omega_{peak} = 1.875 \cdot 1.6$). This remains safely within the strict physical ceiling of the joint actuators ($\omega_{max} = 3.27 \text{ rad/s}$).

However, if the threshold were artificially lowered to 0.15 seconds to mitigate the air-time penalty, the required average speed would increase to 2.66 rad/s ($\omega_{avg} = \frac{0.4}{0.15}$). Applying the exact same formula, this demands a peak velocity of 5.0 rad/s ($\omega_{peak} = 1.875 \cdot 2.66$). Because 5.0 rad/s clearly exceeds the 3.27 rad/s absolute mechanical ceiling, any attempt to execute this shortened swing phase exceeds the strict kinematic velocity ceilings required to maintain safe, continuous operation. Therefore, the 0.25-second threshold is not an arbitrary software choice, but

rather the absolute minimum viable aerial duration required to keep the commanded kinematics safely within the strict mechanical safety margins.

Conversely, artificially raising the threshold above the 0.25-second baseline simply shifts the kinematic demand from the swing phase to the stance phase. If the policy extends the swing duration to satisfy a higher threshold, it must simultaneously compress the stance phase to maintain a stable, standard-cadence trot — as the velocity reward function explicitly penalizes any deviation from the 0.6 m/s target, preventing the policy from simply reducing speed to accommodate a longer cycle.

To illustrate this concretely, extending the swing to 0.35 seconds compresses the stance to 0.15 seconds. Under the Impulse-Momentum Theorem ($F\Delta t = m\Delta v$), the minimum impulsive force required to maintain 0.6 m/s forward momentum within this compressed window is $F = (64.36 \cdot 0.6) / 0.15 = 257.4 \text{ N}$ horizontally. Simultaneously, each stance leg must support its share of the 64.36 kg system weight vertically: $F_z = (64.36 \cdot 9.81) / 2 = 315.0 \text{ N}$. Because these two force components act in perpendicular directions — one horizontal and one vertical — the resultant force each stance leg must sustain is their vector sum: $\sqrt{128.7^2 + 315.0^2} \approx 341 \text{ N}$, where 128.7 N is the per-leg share of the forward impulse ($257.4 / 2$). Applied across the 0.745 m effective limb radius under idealised perpendicular geometry, this yields an indicative upper-bound joint torque of approximately 254 Nm — exceeding the 229.0 Nm continuous limit enforced by the simulation's `dof_torques_12` penalty architecture. Because this penalty scales quadratically, this exceedance generates a negative reward gradient that rapidly outweighs the

linear velocity reward, rendering higher thresholds computationally non-viable within the training environment.

Ultimately, the 0.25-second baseline represents the precise operating boundary identified through kinematic analysis of the hardware specifications — the minimum aerial duration at which the commanded kinematics remain within the mechanical safety margins of the actuator design. Within the current training configuration, the policy consistently stabilized below this threshold despite explicit reward incentive, suggesting that further adjustment in either direction would drive the system outside its viable kinematic envelope.

4.3.3 Postural Adaptation and Dimensionality Reduction

The postural regulation data detailed in Section 4.2.4 highlights a fundamental challenge in synthesizing unconstrained reinforcement learning policies: the tendency for artificial agents to converge on biomechanically stable, yet operationally unviable, local minima. As evidenced by the severe initial deficit in the hip-roll penalty logs, the early unconstrained policy maintained the commanded forward translation, but achieved this via a suboptimal, asymmetric postural adaptation. The untrained agent splayed its front limbs outward into a moderately widened stance, while sweeping its hind limbs asymmetrically to act as a continuous counterbalance.

From a physics perspective, this posture is a rational mathematical exploit. By driving the front hip-roll actuators outward, the agent artificially widened its coronal base of support. This moderate increase in lateral stability allowed the robot to march forward without having to master the precise, high-frequency dynamic balance required for a standard, parallel trot. However, while

this asymmetric posture satisfies the mathematical velocity rewards of the simulation, it violates the spatial constraints of the physical target domain. A widened, asymmetrical footprint is spatially inefficient and incompatible with the narrow corridors and dense debris fields characteristic of nuclear decommissioning and industrial inspection tasks.

Therefore, the implementation of the custom Coronal Plane Constraint (`hip_posture`) acts as a mandatory spatial regularization. The efficacy of this constraint is clearly visible in the training logs. During the first 300 iterations, the untrained agent heavily explored the penalized lateral skew, resulting in a severe accumulation of negative penalties. However, the rigid mathematical stricture successfully suppressed this behaviour. The metric rapidly corrected and stabilized at a marginal residual value near zero, confirming the robot successfully suppressed the extreme postural anomalies while tolerating only the minimal dynamic sway necessary for high-frequency forward translation.

This postural stability is further supported by the chassis orientation metric, which safely converges to a near-zero baseline. Crucially, this metric does not indicate an artificially rigid torso—which is kinematically impossible during a dynamic quadrupedal trot. Instead, the stabilization of this penalty demonstrates that the policy successfully bounds the chassis's pitch and roll within safe mathematical tolerances. The agent learned to aggressively mitigate severe, destabilizing orientations, confining the torso's movement exclusively to the rhythmic lateral weight-shifting required to sustain continuous forward locomotion.

Visual telemetry of the converged policy confirms the success of this dimensionality reduction, while simultaneously revealing a highly optimized kinematic compromise. The agent abandoned the lateral splay, returning the front track width to a nominal, mechanically viable stance. However, it did not adopt a perfectly rigid, parallel alignment ($\parallel$) in the rear. Instead, the hind limbs converged on a subtle inward bias (forming a slight $\backslash /$ geometry), keeping the rear feet closely gathered under the chassis without crossing the sagittal midline or collapsing into self-collision.

This specific convergent posture is not a failure of the constraint but an emergent stabilising adaptation consistent with established quadrupedal biomechanics. By converging the hind feet slightly inward toward the sagittal midline, the policy reduces the lateral moment arm during the single-leg stance phase of the trot cycle. This brings the chassis centre of mass projection closer to the support polygon during diagonal leg transitions, reducing the rotational demand on the hip actuators and improving the stability margin during high-frequency weight transfer. The posture therefore represents a kinematically efficient solution that satisfies both the Coronal Plane Constraint and the dynamic stability requirements of continuous forward locomotion.

4.3.4 Impulse Dissipation and Dynamic Robustness

The time-series telemetry and visual evaluations detailed in Section 4.2.5 demonstrate the policy's capacity for dynamic disturbance rejection. As verified in the recorded trials, the robot is capable of surviving a severe 1.5 m/s external impulse. While the system successfully absorbs the impact and re-establishes its target velocity, the underlying mechanics of this recovery rely on a specific combination of software regularization and learned dynamic behaviour.

First, the immediate dissipation of the kinetic energy is heavily mediated by the baseline Proportional-Derivative (PD) actuator parameters established in Section 3.2.1. When the external impulse violently accelerates the chassis, the derivative damping term ($K_d = 50.0$) reacts to the sudden velocity spike by applying an aggressive, proportional braking torque. This viscous damping effectively bleeds off a significant portion of the injected kinetic energy through the simulated actuators before it can fully destabilize the robot's center of mass.

Second, the robot's physical survival relies on the policy's learned capacity for dynamic limb repositioning. Rather than passively absorbing the impact, the agent actively intercepts the ground with its swinging limbs, dynamically repositioning its feet to re-establish support beneath the shifting centre of mass. This high-frequency stepping response — synthesized entirely through the domain randomization and penalty architecture detailed in Chapter 3 — prevents catastrophic capsizing and channels the dissipated momentum back into forward translation. Visual telemetry confirms that the policy generalises well beyond steady-state locomotion, demonstrating active disturbance rejection that is critical for deployment in the unstructured industrial environments this system targets.

However, as confirmed by qualitative visual telemetry, this dynamic recovery is an emergent survival mechanism rather than a fully deterministic outcome. While the time-series data definitively proves the system possesses the capability to successfully dissipate the 1.5 m/s impulse and recover its target velocity, the robot still exhibits susceptibility to capsizing failures under the exact same perturbation. Because the frequency of these failures was observed

qualitatively rather than formally quantified, this variance indicates that the system's ability to dynamically reposition its limbs is highly dependent on the phase of the gait cycle at the exact moment of impact. Therefore, while this recovery strategy demonstrates reliable recovery on a uniform flat plane, future work must conduct a rigorous statistical analysis to isolate the specific kinematic edge-cases that trigger these failures, and critically evaluate how this robustness scales when transferred to real-world industrial environments.

5. CONCLUSIONS

5.1 Summary

This dissertation investigated the synthesis of a hardware-viable quadrupedal locomotion policy using Deep Reinforcement Learning (DRL) within a rigorous simulation-first workflow. The core problem addressed was the "reality gap"—the tendency for artificial agents to synthesize mathematically optimal but mechanically destructive gaits that fail upon physical deployment. To mitigate this risk for a custom 64.36 kg quadruped intended for hazardous environment inspection, a high-fidelity digital twin was constructed within NVIDIA Isaac Lab.

Rather than relying on unconstrained learning, the methodology prioritized strict physical realism. The Proximal Policy Optimization (PPO) agent was subjected to comprehensive domain randomization and rigidly bounded by the exact kinematic and dynamic ceilings of the physical platform's motor-gearbox assemblies. The empirical results demonstrated that the policy successfully converged without experiencing catastrophic collapse, synthesizing a stable, continuous forward trot.

The kinematic and time-series analysis demonstrated that the control policy successfully learned to operate within the absolute mechanical boundaries of the hardware. Faced with an extreme 0.6 m/s kinematic stress test at the upper velocity boundary of the domain randomization range, the agent converged on a stable, continuous gait that strictly respects the actuator velocity and torque limits enforced by the digital twin. Ultimately, these results validate the efficacy of a constraint-heavy digital twin as a pre-deployment diagnostic tool: by enforcing strict hardware safety margins in simulation, the framework successfully forged a robust, energy-efficient

locomotion controller capable of maintaining stable forward translation and rejecting severe external disturbances, providing strong simulation-based evidence of mechanical viability for the physical chassis prior to real-world deployment.

5.2 Discussion

This dissertation provides a validated, practical framework for bridging the reality gap in quadrupedal reinforcement learning for highly custom hardware. Historically, deploying deep reinforcement learning policies onto physical robots has been hindered by the necessity of relying on pre-existing, commercial platforms or risking catastrophic damage to unproven prototypes. By successfully engineering a rigorous digital twin within NVIDIA Isaac Lab, this research demonstrates that custom, high-inertia robotic platforms can be safely and dynamically stress-tested *in silico*. This dissertation establishes the foundational control architecture for this specific robot, contributing to the broader field of applied robotics by proving that hardware safety and AI gait synthesis can be simultaneously achieved prior to physical manufacturing.

5.2.1 Contributions

This dissertation provides several concrete, foundational contributions to the development of the custom quadruped platform:

- **First-of-its-Kind Control Architecture:** Developed the very first custom code framework capable of controlling this specific, previously untested 64.36 *kg* chassis.

Prior to this work, the robot existed solely as a passive mechanical design; this research provides its first functional, AI-driven locomotion system.

- **End-to-End Digital Twin Pipeline:** Successfully translated a static CAD assembly into a highly interactive, physics-compliant Universal Scene Description (USD) entity, establishing a fully functional cyber-environment for a robot that had never been simulated before.
- **Baseline Locomotion Synthesis:** Established the foundational flat-terrain trotting policy, proving mathematically and visually that the digital twin can successfully synthesize continuous dynamic motion within strict hardware boundaries.

5.2.2 Impact

The primary impact of this dissertation lies in its complete transformation of the hardware development lifecycle for this project. By establishing a high-fidelity *in silico* testing environment, this framework effectively decouples software control development from physical hardware availability. It allows engineers to conduct highly dynamic, destructive experimentation on the robot before the physical hardware is ever assembled.

Crucially, this unprecedented codebase provides a modular testing ground for all future iterations of the platform. Engineers can now easily tweak CAD geometries, adjust mass distributions, or swap virtual actuators to test if a proposed design change can yield a robust control policy. This shifts the engineering paradigm from a costly "build-and-test" approach to a highly

efficient "simulate-then-build" model, guaranteeing that the hardware and AI software combination is physically viable before expending financial resources on real-world manufacturing.

5.2.3 Significance

This research is highly significant as it demonstrates a cutting-edge application of state-of-the-art simulation ecosystems—specifically the newly introduced NVIDIA Isaac Lab framework powered by the underlying Isaac Sim physics engine—for bespoke, heavy-duty industrial robotics. While much of the existing Deep Reinforcement Learning literature relies on standard, pre-solved commercial platforms (such as the Unitree Go1 or ANYmal), this work proves that these novel technologies can be successfully leveraged to grow an autonomous control policy entirely from scratch for a completely unique, unproven mechanical configuration.

By delivering the first operational control framework for this specific hardware, this dissertation expands the proven utility of the Isaac Lab ecosystem and provides the mandatory computational foundation for all future developers on this project. The digital twin established here serves as the crucial stepping stone that will ultimately allow the robot to transition from a simulated cyber-environment to active, physical deployment.

5.3 Future Work

While this dissertation successfully provides the foundational digital twin and baseline locomotion policy for the physical quadruped platform, further research is required to achieve full

autonomous deployment in hazardous environments. Transitioning to real-world application necessitates the following sequential developments:

- **Hardware Calibration and Direct Sim-to-Real Transfer:** The immediate next step is deploying the trained policy onto the physical prototype. As established in Section 3.2.1, while the physical PD controllers will naturally draw the transient current spikes required to overcome the true 14 *Nm* static stiction of the gearboxes, empirical fine-tuning of the physical tracking gains (K_p and K_d) is still required. This final calibration will ensure that as the hardware powers through this physical stiction, the resulting high-frequency motor response remains smooth and free of micro-oscillations during real-world trajectory tracking.
- **Topographical Curriculum and Exteroceptive Integration:** Once a rigorous statistical analysis of impact-recovery edge-cases is conducted to guarantee dynamic stability, the training curriculum must be expanded. Future policies should introduce complex procedural terrain generation and integrate exteroceptive sensors, such as LiDAR or onboard depth cameras. This will transition the control architecture from purely reactive dynamic recovery to proactive trajectory planning, establishing the final prerequisite for navigating the unstructured debris fields of nuclear decommissioning zones.

REFERENCES

- Alexander, R. M. (2002). *Principles of Animal Locomotion*. Princeton University Press.
- Ali, M., Giri, S., Liu, S., & Yang, Q. (2025). Digital Twin Synchronization: Bridging the Sim-RL Agent to a Real-Time Robotic Additive Manufacturing Control [Preprint]. *arXiv*. <https://arxiv.org/abs/2501.18016>
- Aljalbout, E. et al. (2025). The Reality Gap in Robotics: Challenges, Solutions, and Best Practices [Preprint]. *arXiv*. <https://arxiv.org/abs/2510.20808>
- Angarola, M.P., Affonso, F., & Becker, M. (2025). Learning Terrain-Specialized Policies for Adaptive Locomotion in Challenging Environments [Preprint]. *arXiv*. <https://arxiv.org/abs/2509.20635>
- Bellegarda, G., Chen, Y., Liu, Z., & Nguyen, Q. (2021). Robust High-Speed Running for Quadruped Robots via Deep Reinforcement Learning [Preprint]. *arXiv*. <https://arxiv.org/abs/2103.06484>
- Beranek, R., Karimi, H.R., & Ahmadi, M. (2022). A Behavior-Based Reinforcement Learning Approach to Control Walking Bipedal Robots Under Unknown Disturbances. *IEEE/ASME Transactions on Mechatronics*, 27(5), pp. 2710-2721
- Biagiotti, L., & Melchiorri, C. (2008). *Trajectory Planning for Automatic Machines and Robots*. Springer Berlin, Heidelberg. ISBN: 978-3-540-85628-3. (Chapter 3, Section 3.2.3)
- Bjelonic, M., Tischhauser, K., & Hutter, M. (2025). Towards bridging the gap: Systematic sim-to-real transfer for diverse legged robots [Preprint]. *arXiv*. <https://arxiv.org/abs/2509.06342>
- Bogdanovic, M., Khadiv, M., & Righetti, L. (2022). Model-free reinforcement learning for robust locomotion using demonstrations from trajectory optimization. *Frontiers in Robotics and AI*, 9:854212
- Chen, G. et al. (2022). Quadruped Capturability and Push Recovery via a Switched-Systems Characterization of Dynamic Balance [Preprint]. *arXiv*. <https://arxiv.org/abs/2201.11928>
- Chen, S. et al. (2024). SLR: Learning Quadruped Locomotion without Privileged Information [Preprint]. *arXiv*. <https://arxiv.org/abs/2406.04835>
- Duan, H. et al. (2025). Architecture Is All You Need: Diversity-Enabled Sweet Spots for Robust Humanoid Locomotion [Preprint]. *arXiv*. <https://arxiv.org/abs/2510.14947>
- Gurram, M. (2024). Reinforcement Learning for Quadrupedal Locomotion: Current Advancements and Future Perspectives [Preprint]. *arXiv*. <https://arxiv.org/abs/2410.10438>

Hu, J. et al. (2025). Impact of Static Friction on Sim2Real in Robotic Reinforcement Learning [Preprint]. *arXiv*. <https://arxiv.org/abs/2503.01255>

Isaac Lab Team (2025). Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning [Preprint]. *arXiv*. <https://arxiv.org/abs/2511.04831>

Karakasis, C., Poulakakis, I., & Artemiadis, P. (2022). Robust Dynamic Walking for a 3D Dual-SLIP Model under One-Step Unilateral Stiffness Perturbations: Towards Bipedal Locomotion over Compliant Terrain [Preprint]. *arXiv*. <https://arxiv.org/abs/2203.07471>

Khalil, A. et al. (2025). Success in Humanoid Reinforcement Learning under Partial Observation [Preprint]. *arXiv*. <https://arxiv.org/abs/2507.18883>

Kim, J. et al. (2023). Learning Terrain-Adaptive Locomotion with Agile Behaviors by Imitating Animals [Preprint]. *arXiv*. <https://arxiv.org/abs/2308.03273>

Kovalev et al. (2025). Differentiable simulation system identification integrated into reinforcement learning using MuJoCo-XLA [Preprint]. *arXiv*. <https://arxiv.org/abs/2508.04696>

Le, H. et al. (2022). Learning to Constrain Policy Optimization with Virtual Trust Region. *Advances in Neural Information Processing Systems (NeurIPS 2022)*

Lee, J., Hwangbo, J., & Hutter, M. (2019). Robust Recovery Controller for a Quadrupedal Robot using Deep Reinforcement Learning [Preprint]. *arXiv*. <https://arxiv.org/abs/1901.07517>

Lee, J., Hwangbo, J., Wellhausen, L., Koltun, V., & Hutter, M. (2020). Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 5(47), eabc5986.

Li, Z. et al. (2021). Reinforcement Learning for Robust Parameterized Locomotion Control of Bipedal Robots. *Proceedings of IEEE ICRA 2021*

Liu, M., Pathak, D., & Agarwal, A. (2025). LocoFormer: Generalist Locomotion via Long-context Adaptation [Preprint]. *arXiv*. <https://arxiv.org/abs/2509.23745>

Liu, Q. et al. (2024). MASQ: Multi-Agent Reinforcement Learning for Single Quadruped Robot Locomotion [Preprint]. *arXiv*. <https://arxiv.org/abs/2408.13759>

Masuda, S., & Takahashi, M. (2023). Sim-to-Real Transfer of Compliant Bipedal Locomotion on Torque Sensor-Less Gear-Driven Humanoid [Preprint]. *arXiv*. <https://arxiv.org/abs/2204.03897>

Mirza, K., & Singh, A. (2025). Imitation learning for legged robot locomotion: a survey. *Frontiers in Robotics and AI*, 12:1678567

Mishra, P. et al. (2025). HACL: History-Aware Curriculum Learning for Fast Locomotion [Preprint]. *arXiv*. <https://arxiv.org/abs/2505.18429>

Mock, A., & Muknahallipatna, S. (2023). A Comparison of PPO, TD3 and SAC Reinforcement Algorithms for Quadruped Walking Gait Generation. *Journal of Intelligent Learning Systems and Applications*, 15, pp. 195-206

Nahrendra, I.M.A., Yu, B., & Myung, H. (2023). DreamWaQ: Learning Robust Quadrupedal Locomotion With Implicit Terrain Imagination via Deep Reinforcement Learning [Preprint]. *arXiv*. <https://arxiv.org/abs/2301.10602>

Nguyen, T. et al. (2025). Informed Asymmetric Actor-Critic: Leveraging Privileged Signals Beyond Full-State Access [Preprint]. *arXiv*. <https://arxiv.org/abs/2509.26000>

Nguyen, T. et al. (2025b). 'Extended Hybrid Zero Dynamics for Bipedal Walking of the Kneeless Robot SLIDER', *arXiv*:2504.01165.

Patel, R. et al. (2025). Benchmarking Model Predictive Control and Reinforcement Learning Based Control for Legged Robot Locomotion in MuJoCo Simulation [Preprint]. *arXiv*. <https://arxiv.org/abs/2501.16590>

Peng, X. B., Andrychowicz, M., Zaremba, W., & Abbeel, P. (2018). Sim-to-Real Transfer of Robotic Control with Dynamics Randomization. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3803-3810.

Plozza, D. et al. (2025). Robust Reinforcement Learning-Based Locomotion for Resource-Constrained Quadrupeds with Exteroceptive Sensing [Preprint]. *arXiv*. <https://arxiv.org/abs/2505.12537>

Roux, C. et al. (2025). Constrained Reinforcement Learning for Unstable Point-Feet Bipedal Locomotion Applied to the Bolt Robot [Preprint]. *arXiv*. <https://arxiv.org/abs/2508.02194>

Sakakibara, Y., Kan, K., Hosoda, Y., Hattori, M., & Fujie, M. (1990). Foot trajectory for a quadruped walking machine. *Proceedings IEEE International Workshop on Intelligent Robots and Systems, Towards a New Frontier in Applications (IROS)*. <https://doi.org/10.1109/IROS.1990.262407>

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms [Preprint]. *arXiv*. <https://arxiv.org/abs/1707.06347>

Sikchi, H. et al. (2020). Attention-Privileged Reinforcement Learning. *Conference on Robot Learning (CoRL)*; *arXiv*:1911.08363

Song, S. et al. (2018). Recurrent Deterministic Policy Gradient Method for Bipedal Locomotion on Rough Terrain Challenge [Preprint]. *arXiv*. <https://arxiv.org/abs/1710.02896>

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). The MIT Press.

Tang, C. et al. (2024). Deep Reinforcement Learning for Robotics: A Survey of Real-World Successes [Preprint]. *arXiv*. <https://arxiv.org/abs/2408.03539>

Tiboni, G., Arndt, K., & Kyrki, V. (2023). DROPO: Sim-to-real transfer with offline domain randomization. *Robotics and Autonomous Systems*, 166, 104432

Wan, H., Li, M., Wu, D., Zhong, Y., Deng, Y., Sun, Z., & Ji, X. (2025). GenTe: Generative Real-world Terrains for General Legged Robot Locomotion Control. *arXiv preprint*, arXiv:2504.09997

Wang, Y. et al. (2024). Key estimation and saliency analysis for robust legged locomotion [Preprint]. *arXiv*. <https://arxiv.org/abs/2403.05868>

Wu, Y. et al. (2025). Learn to Teach: Sample-Efficient Privileged Learning for Humanoid Locomotion over Real-World Uneven Terrain [Preprint]. *arXiv*. <https://arxiv.org/abs/2402.06783>

Yang, Y. et al. (2022). Bayesian Optimization Meets Hybrid Zero Dynamics: Safe Parameter Learning for Bipedal Locomotion Control [Preprint]. *arXiv*. <https://arxiv.org/abs/2203.02570>

Ye, Y. et al. (2022). The Simplest Balance Controller for Dynamic Walking [Preprint]. *arXiv*. <https://arxiv.org/abs/2211.06223>

Yu, Y. et al. (2022). Dynamic Bipedal Turning through Sim-to-Real Reinforcement Learning. *IEEE-RAS Humanoids 2022*

Yue, J. (2023). Learning Locomotion for Legged Robots Based on Reinforcement Learning: A Survey. preprint (PDF)

Zhang, Q. et al. (2025a). 'Distillation-PPO: A Novel Two-Stage Reinforcement Learning Framework for Humanoid Robot Perceptive Locomotion', *arXiv*:2503.08299.

Zhang, Y. et al. (2024). Ask1: Development and Reinforcement Learning-Based Control of a Custom Quadruped Robot [Preprint]. *arXiv*. <https://arxiv.org/abs/2412.08019>

Zhang, Y. et al. (2025b). 'Adversarial Locomotion and Motion Imitation for Humanoid Policy Learning (ALMI)', *arXiv*:2504.14305.

Zhang, Z., Zheng, H., Cheng, H., & Qiu, J. (2020). Leg Locomotion Adaption for Quadruped Robots with Ground Compliance Estimation. *Applied Bionics and Biomechanics*. <https://doi.org/10.1155/2020/8854411>

Zhang, Z. et al. (2022). 3D-SLIP model based dynamic stability strategy for legged robots with impact disturbance rejection. *Scientific Reports*, 12:9937

Zhao, Y. et al. (2024). ZSL-RPPO: Zero-Shot Learning for Quadrupedal Locomotion in Challenging Terrains using Recurrent Proximal Policy Optimization [Preprint]. *arXiv*.
<https://arxiv.org/abs/2403.01928>

Zhou, J. et al. (2023). Combining Model-Predictive Control and Predictive Reinforcement Learning for Stable Quadrupedal Robot Locomotion [Preprint]. *arXiv*.
<https://arxiv.org/abs/2307.07752>

Zhou, X. et al. (2024). RL2AC: Reinforcement Learning-based Rapid Online Adaptive Control for Legged Robot Robust Locomotion. *Robotics: Science and Systems (RSS) 2024*

This is the final page of the Dissertation and was intentionally left blank.